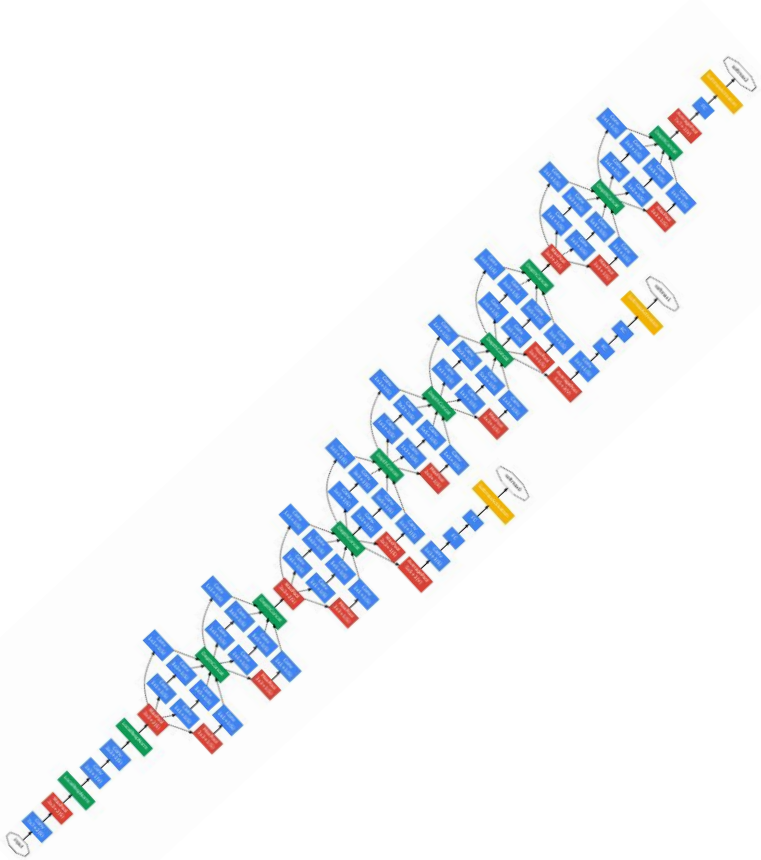
Deep Learning for Computer Vision (Discriminative way)

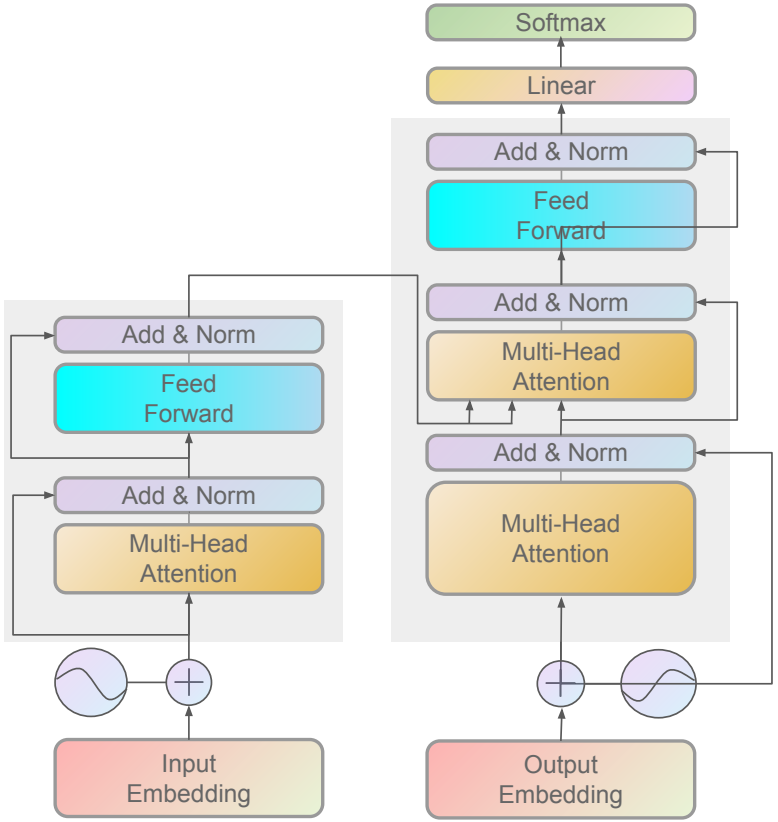
Andrii Liubonko

2025

CV DL fundamentals



Convolutional Neural Networks (CNN)

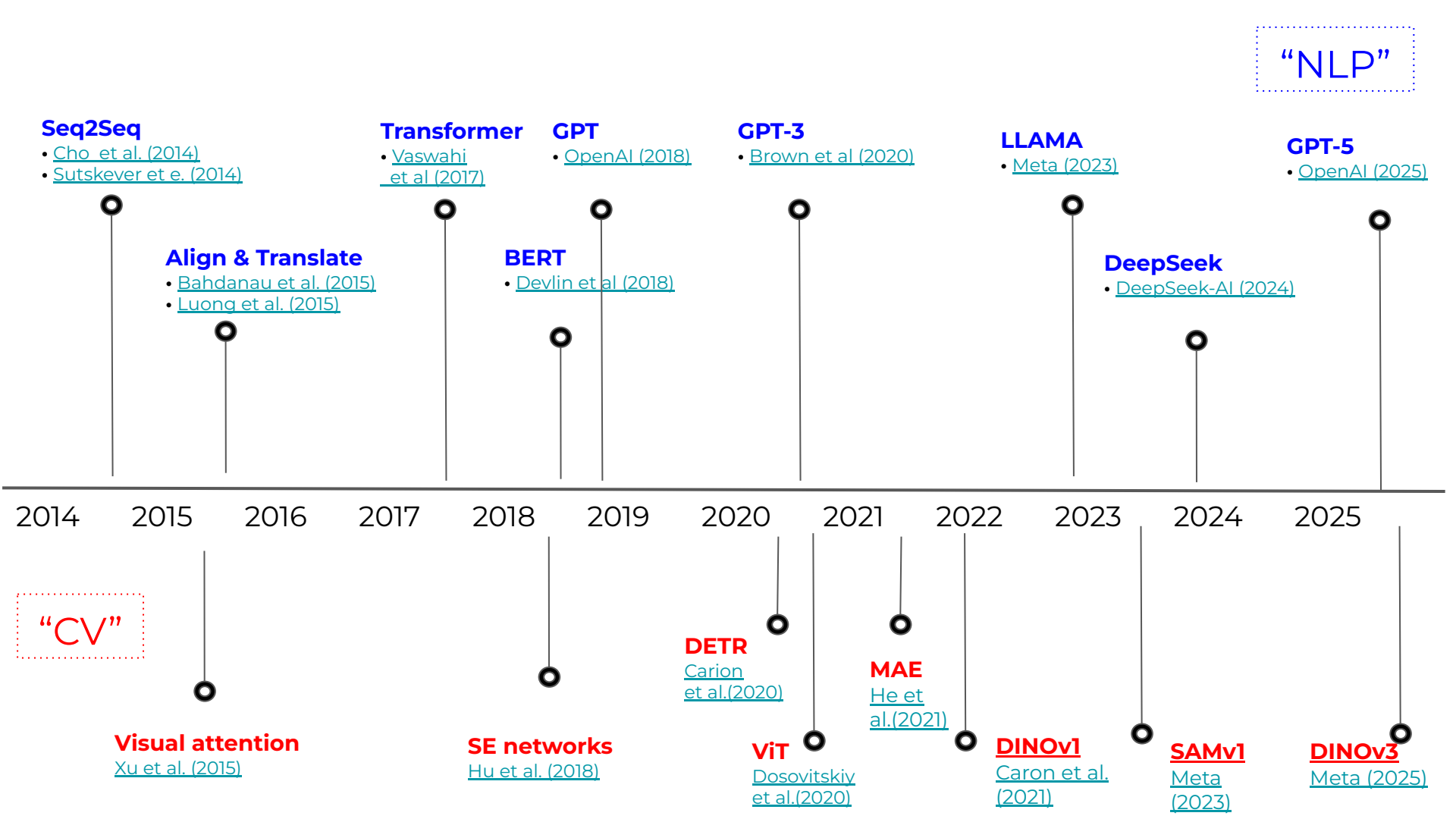


Transformer-based

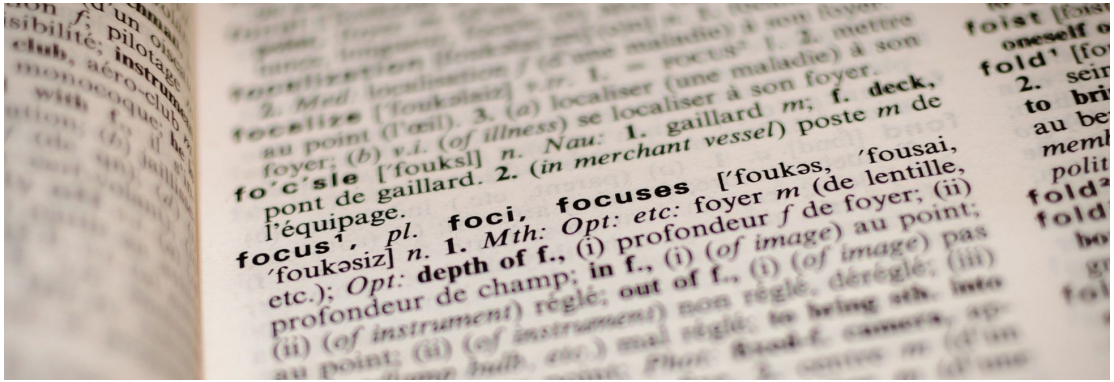
Content of today's lecture

- Attention Intro
  - Seq2seq Attention
  - Self-Attention
  - Transformer

- Attention in CV
  - SE
  - ViT
  - latest developments


"NLP"
Seq2Seq
• Cho et al. (2014)
• Sutskever et e. (2014)
Align & Translate
• Bahdanau et al. (2015)
• Luong et al. (2015)
Transformer
• Vaswahi et al (2017)
BERT
• Devlin et al (2018)
GPT
• OpenAI (2018)
GPT-3
• Brown et al (2020)
LLAMA
• Meta (2023)
DeepSeek
• DeepSeek-AI (2024)
GPT-5
• OpenAI (2025)
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
"CV"
Visual attention
Xu et al. (2015)
SE networks
Hu et al. (2018)
DETR
Carion et al.(2020)
ViT
Dosovitskiy et al.(2020)
MAE
He et al.(2021)
DINOv1
Caron et al. (2021)
SAMv1
Meta (2023)
DINOv3
Meta (2025)

Attention [intro]



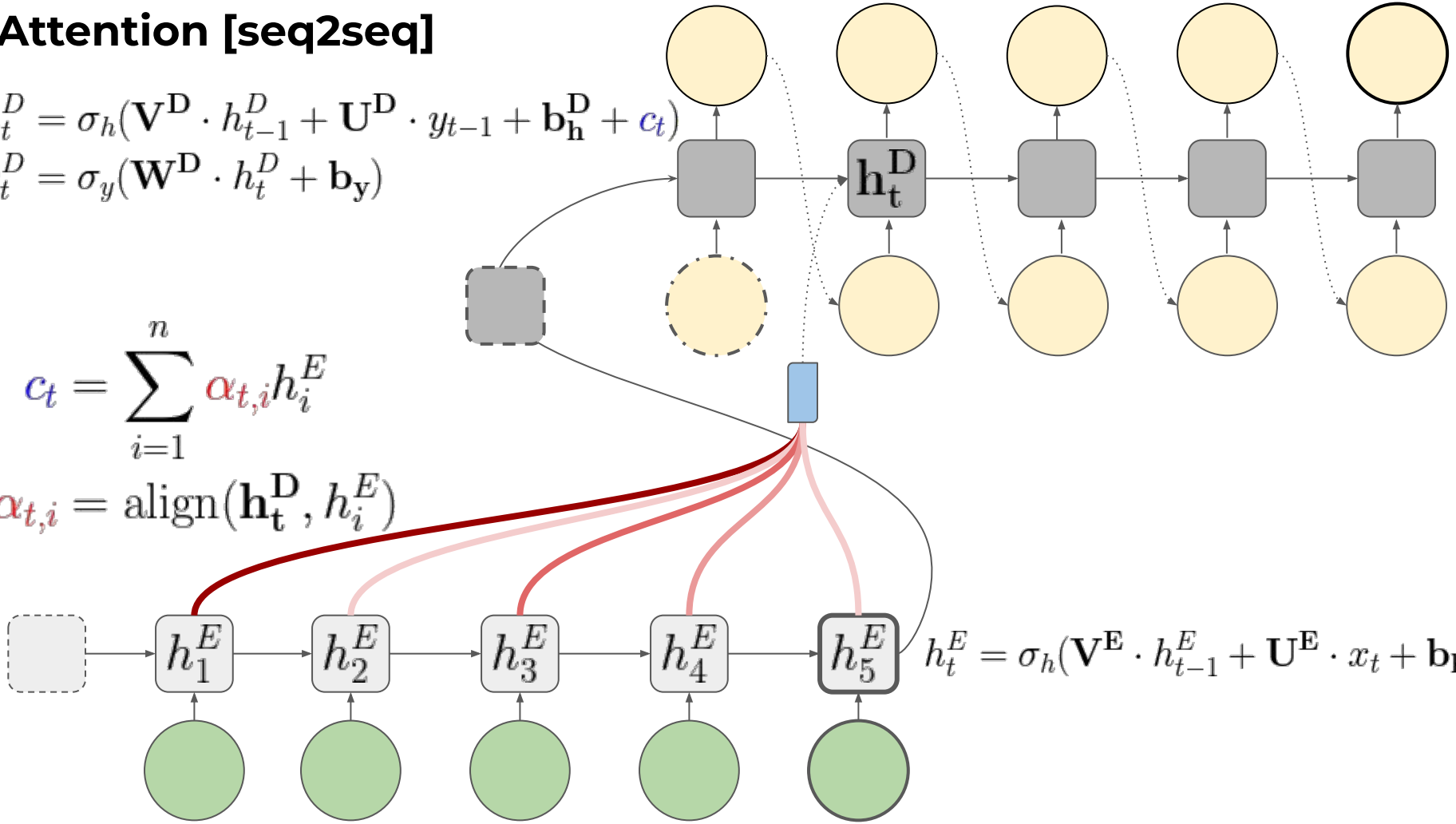
In a nutshell, attention in deep learning can be broadly interpreted as a vector of importance weights: in order to predict or infer one element, such as a pixel in an image or a word in a sentence, we estimate using the attention vector how strongly it is correlated with (or “attends to”) other elements

Attention [seq2seq]

$$h_t^D = \sigma_h(\mathbf{V^D} \cdot h_{t-1}^D + \mathbf{U^D} \cdot y_{t-1} + \mathbf{b_h^D} + c_t)$$
$$y_t^D = \sigma_y(\mathbf{W^D} \cdot h_t^D + \mathbf{b_y})$$

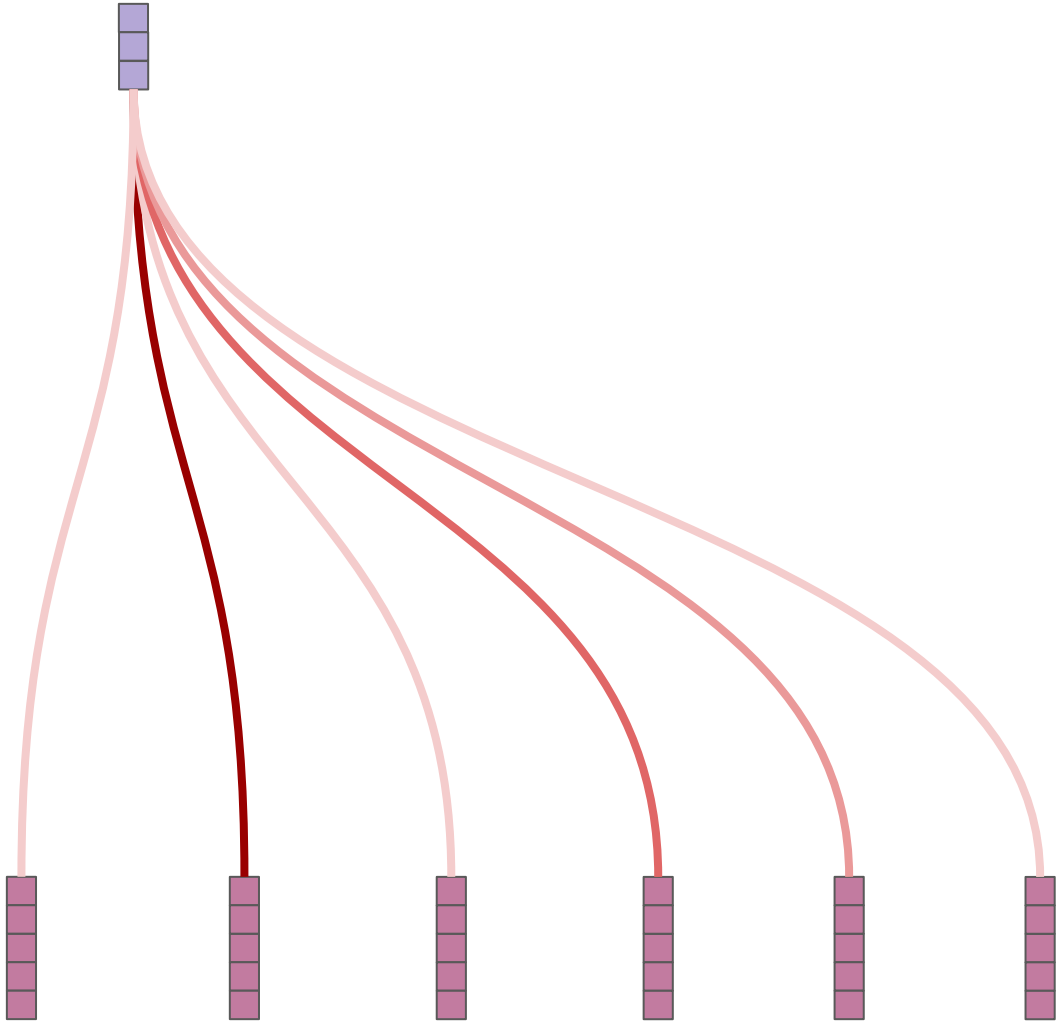
$$c_t = \sum_{i=1}^{n} \alpha_{t,i} h_i^E$$

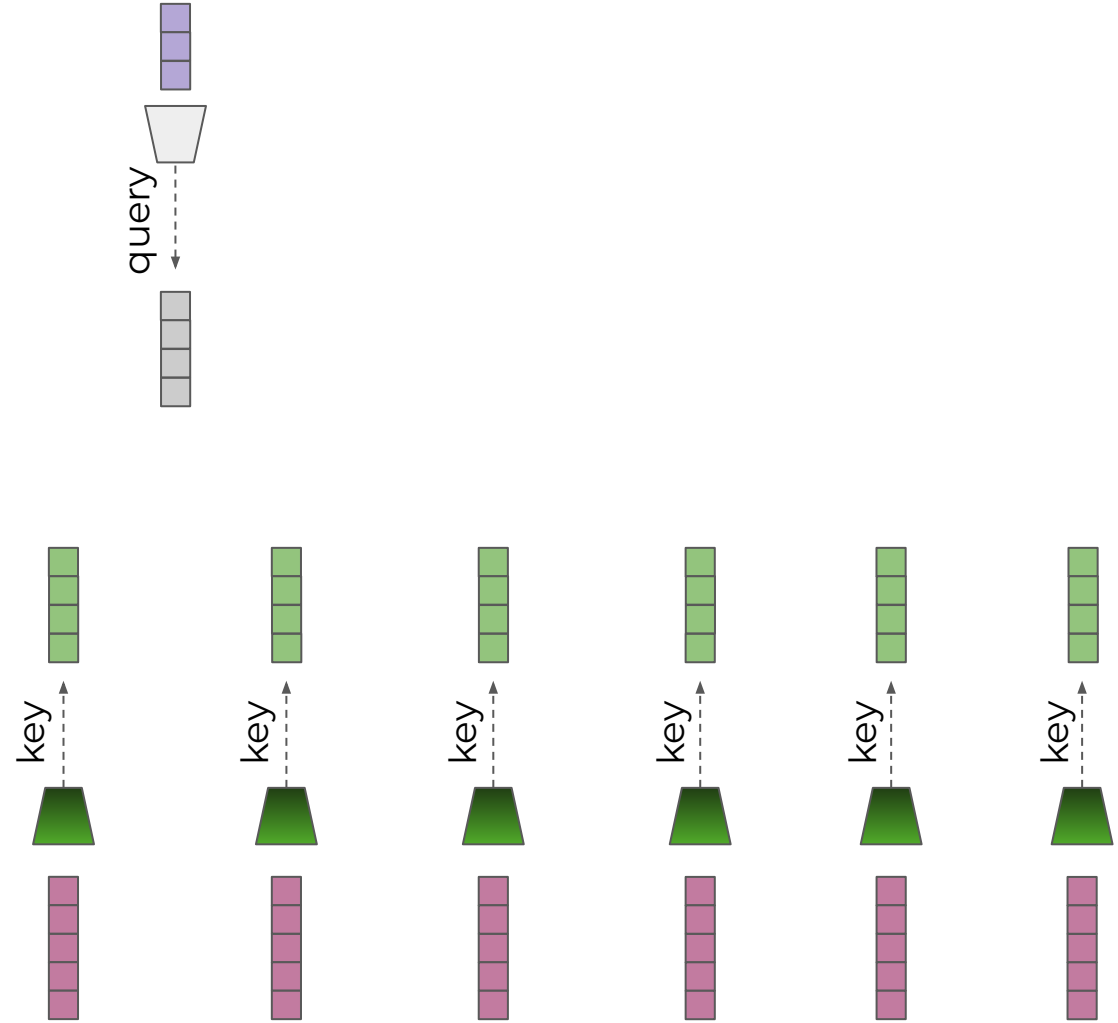
$$\alpha_{t,i} = \text{align}(\mathbf{h_t^D}, h_i^E)$$

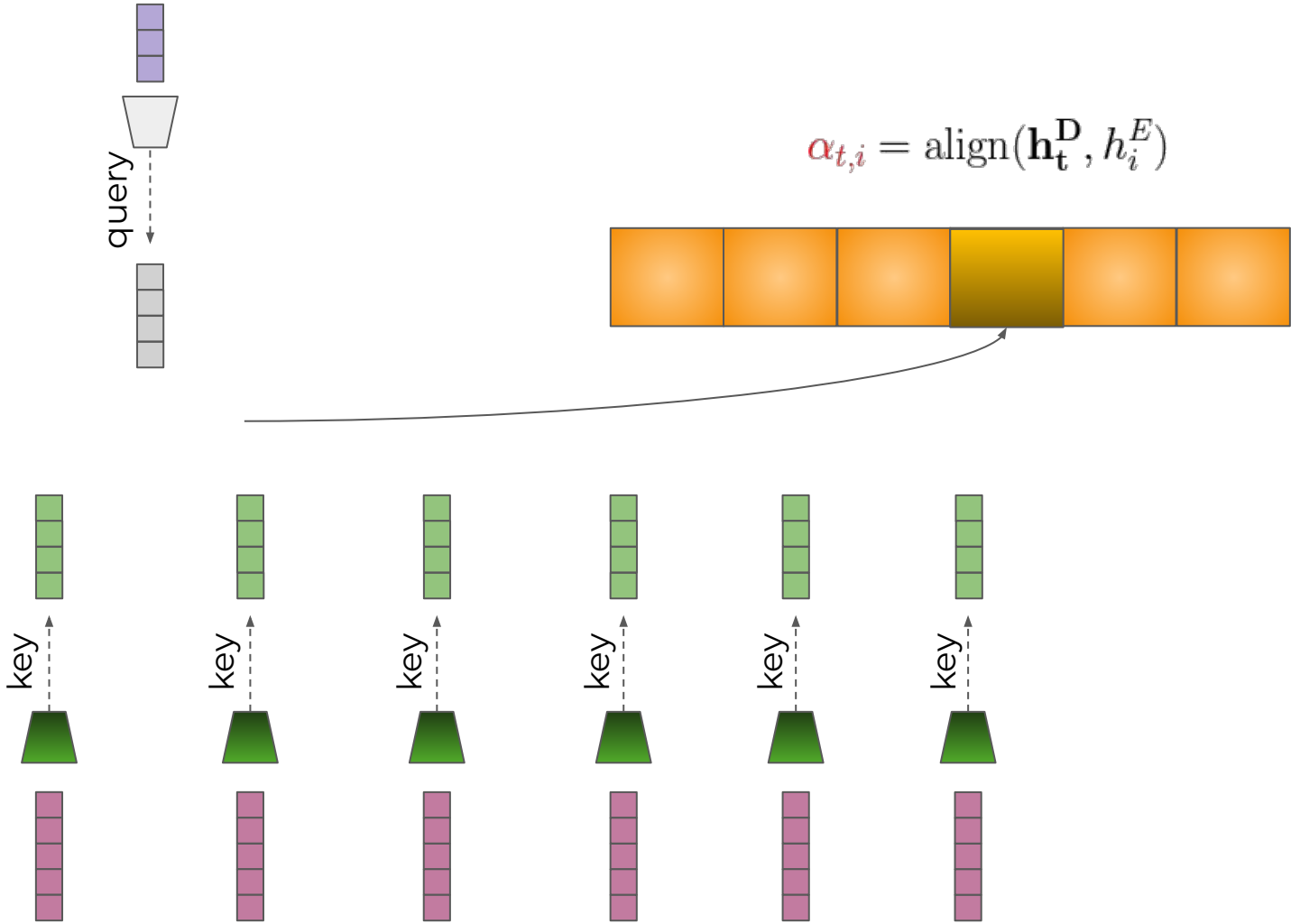


$$h_t^E = \sigma_h(\mathbf{V^E} \cdot h_{t-1}^E + \mathbf{U^E} \cdot x_t + \mathbf{b_h})$$

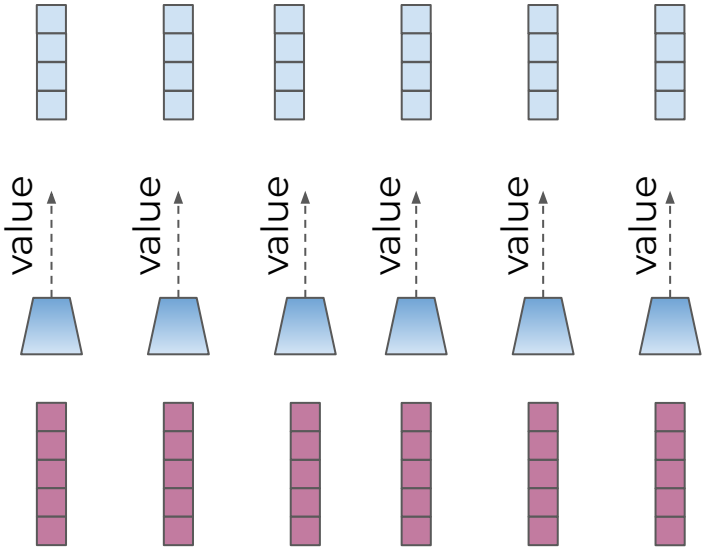
query-key-value abstraction




query
key
key
key
key
key
key


query
$\alpha_{t,i} = \text{align}(\mathbf{h_t^D}, h_i^E)$
key
key
key
key
key
key

$$\alpha_{t,i} = \text{align}(\mathbf{h_t^D}, h_i^E)$$



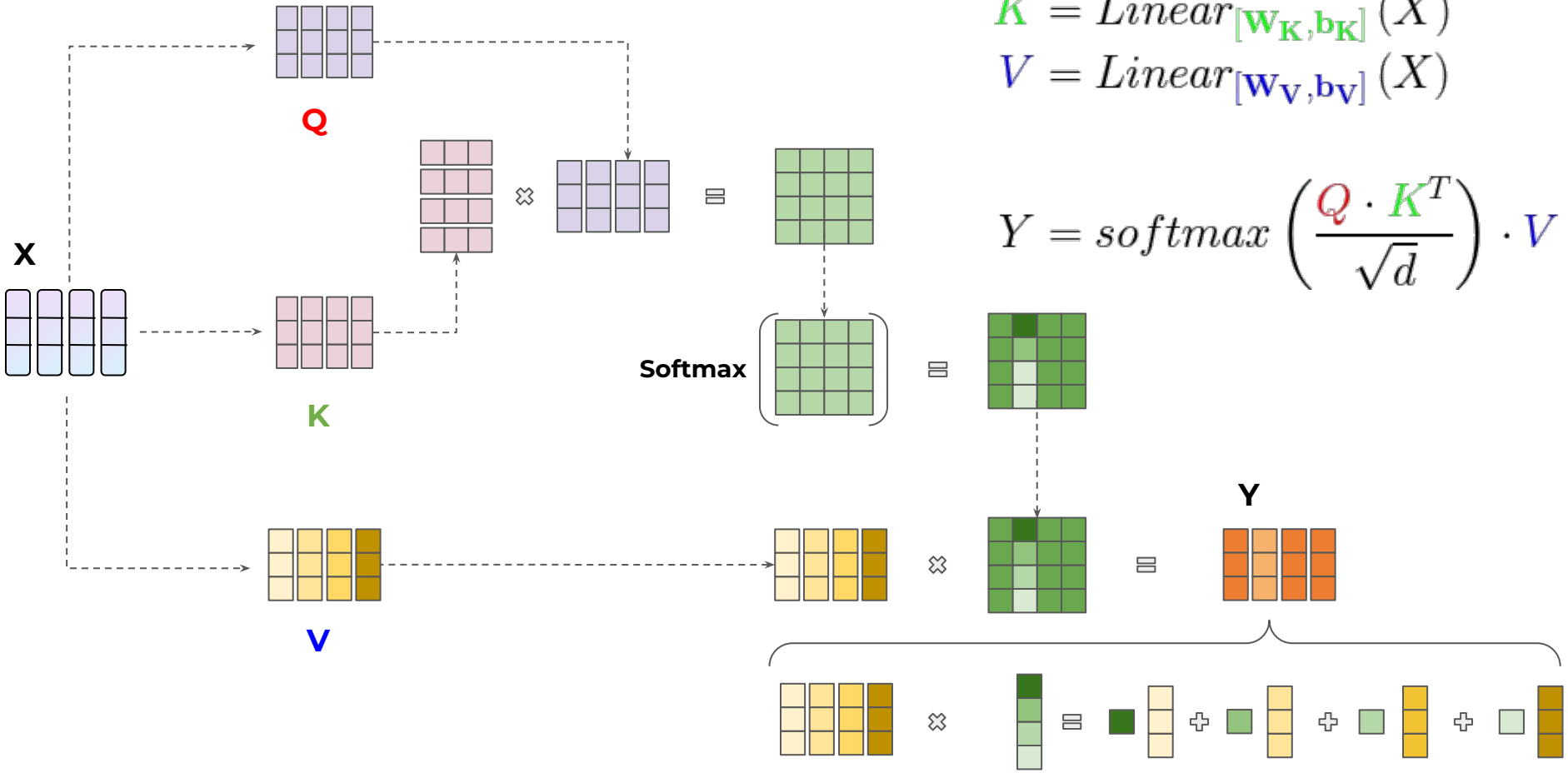
$$c_t = \sum_{i=1}^{n} \alpha_{t,i} h_i^E$$

Self-attention

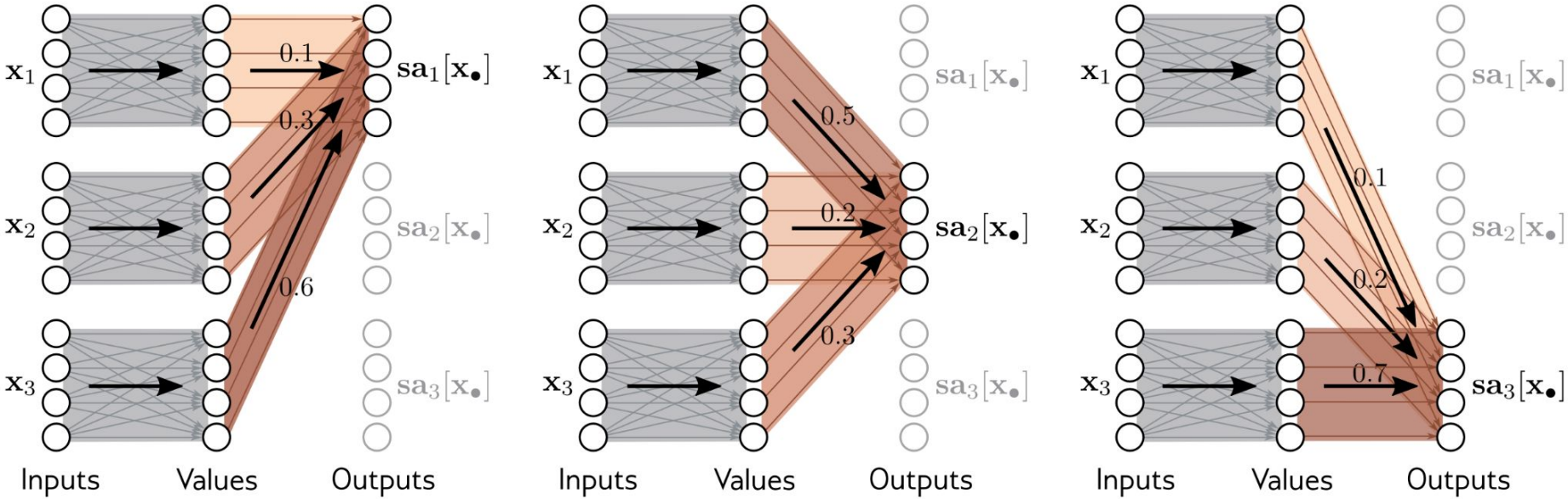
Self-Attention

$$Q = Linear_{[\mathbf{W_Q}, \mathbf{b_Q}]}(X)$$
$$K = Linear_{[\mathbf{W_K}, \mathbf{b_K}]}(X)$$
$$V = Linear_{[\mathbf{W_V}, \mathbf{b_V}]}(X)$$

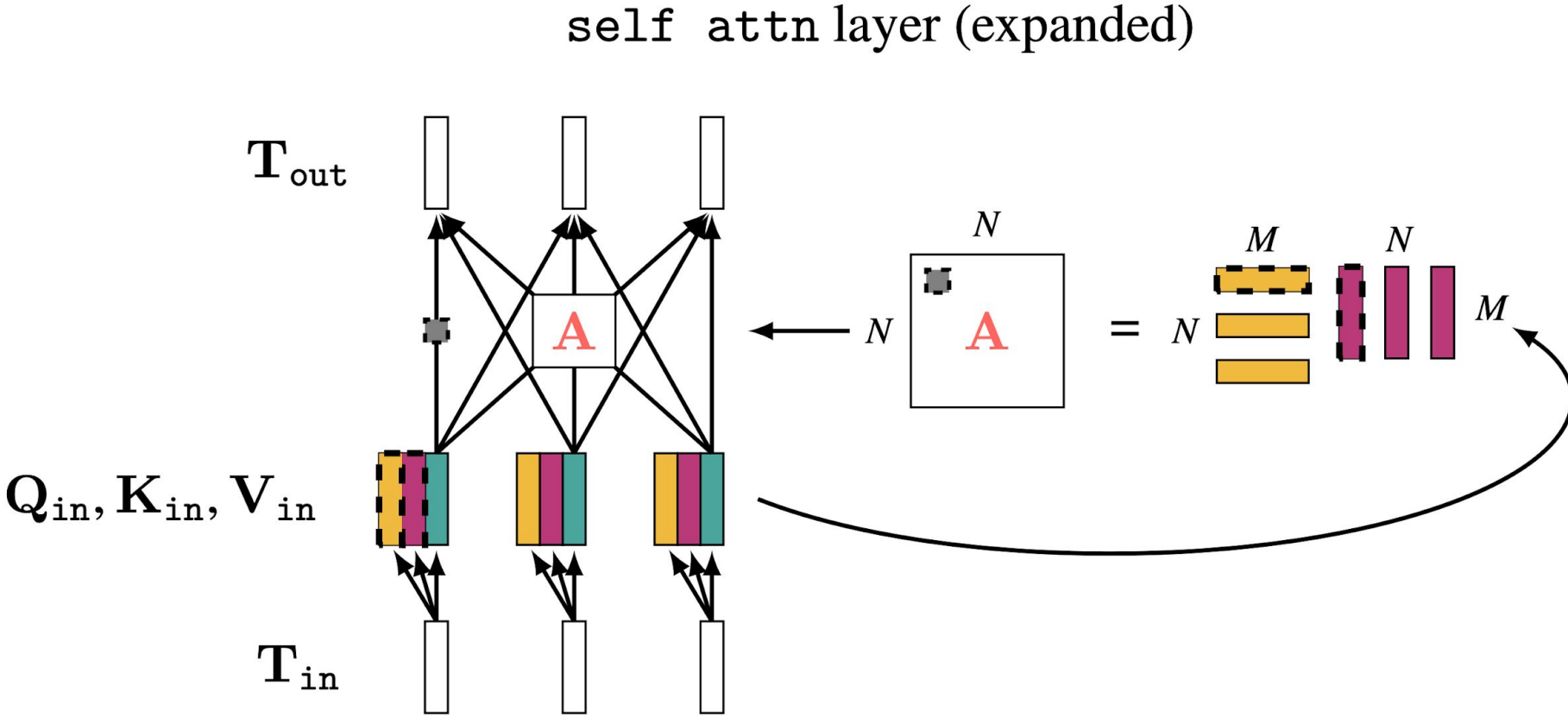
$$Y = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) \cdot V$$



Self-Attention

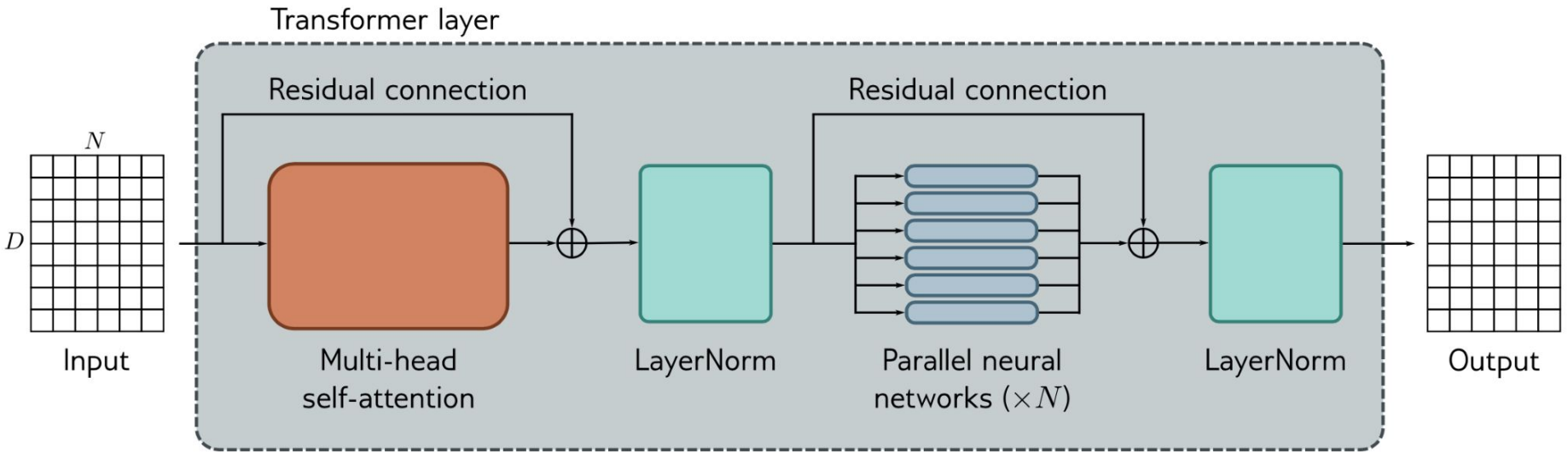


* image from Understanding Deep Learning, book by Simon J.D. Prince, 2023



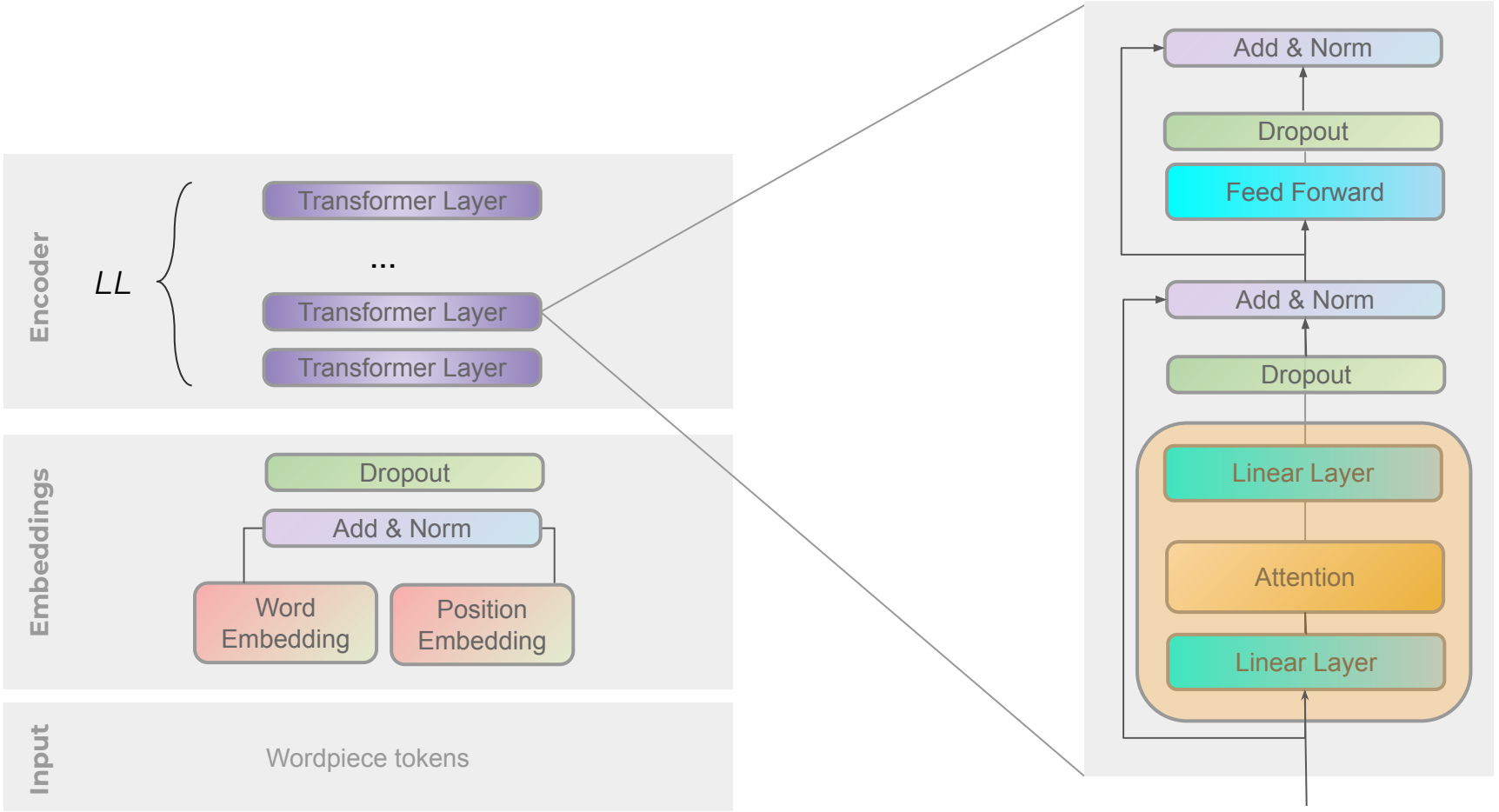
https://visionbook.mit.edu/transformers.html

Transformer layer

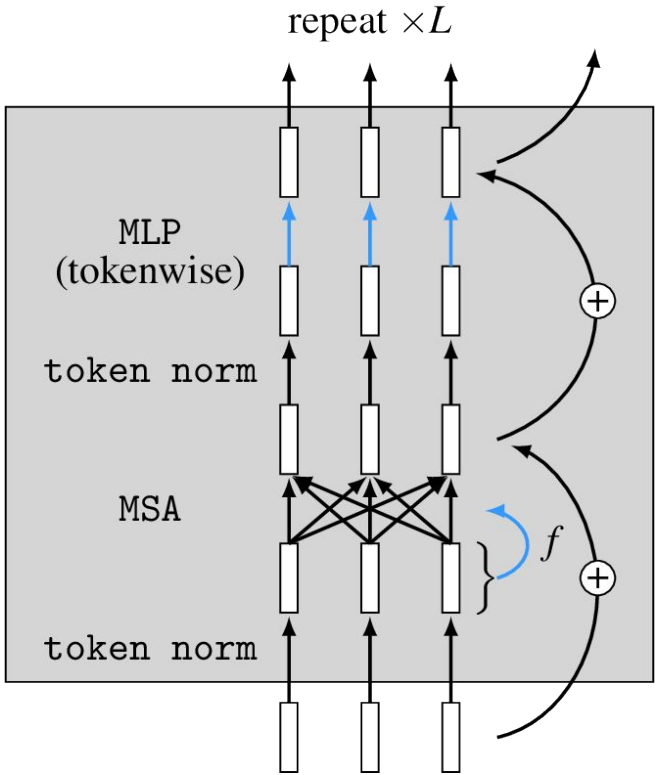


* image from Understanding Deep Learning, book by Simon J.D. Prince, 2023

Transformer

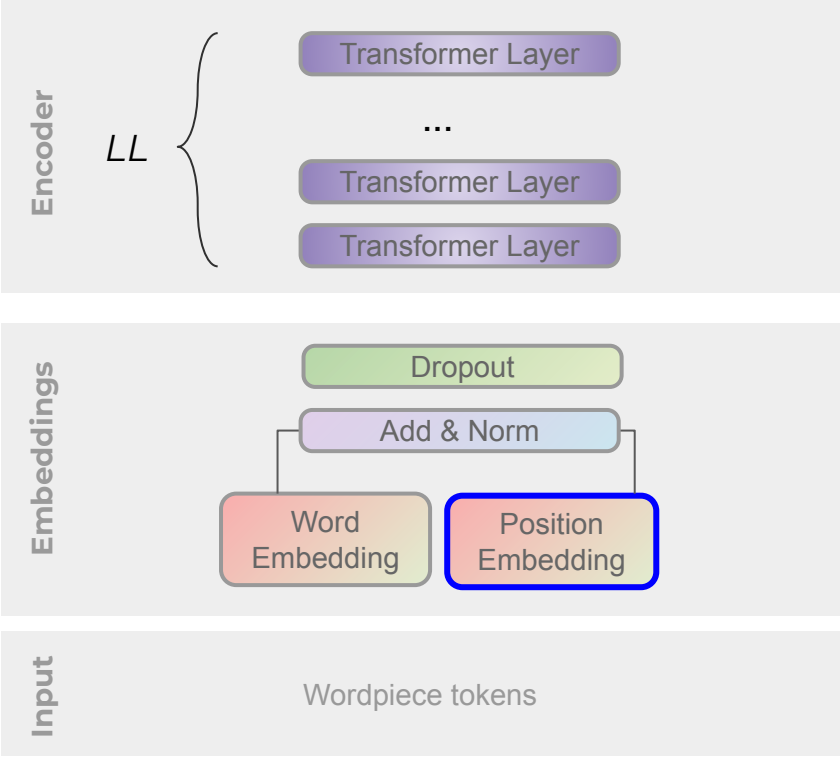


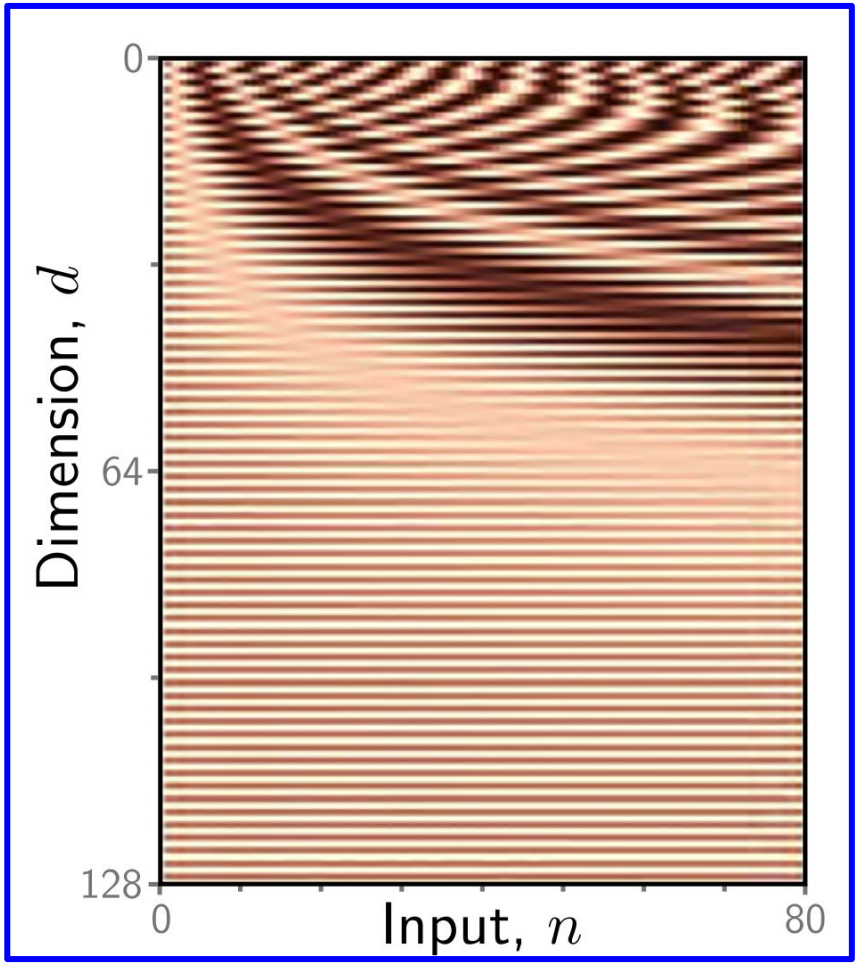
Transformer



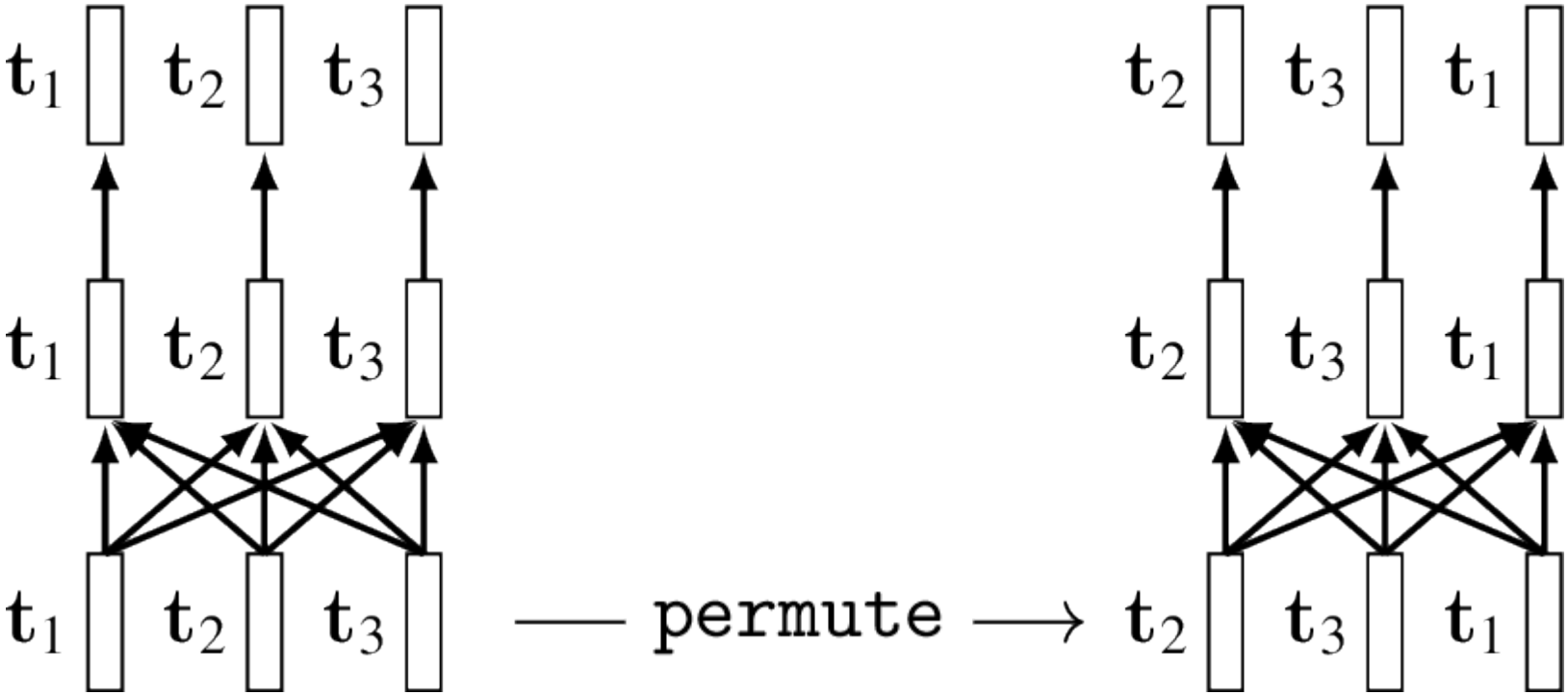
https://visionbook.mit.edu/transformers.html

Transformer [Positional embedding]





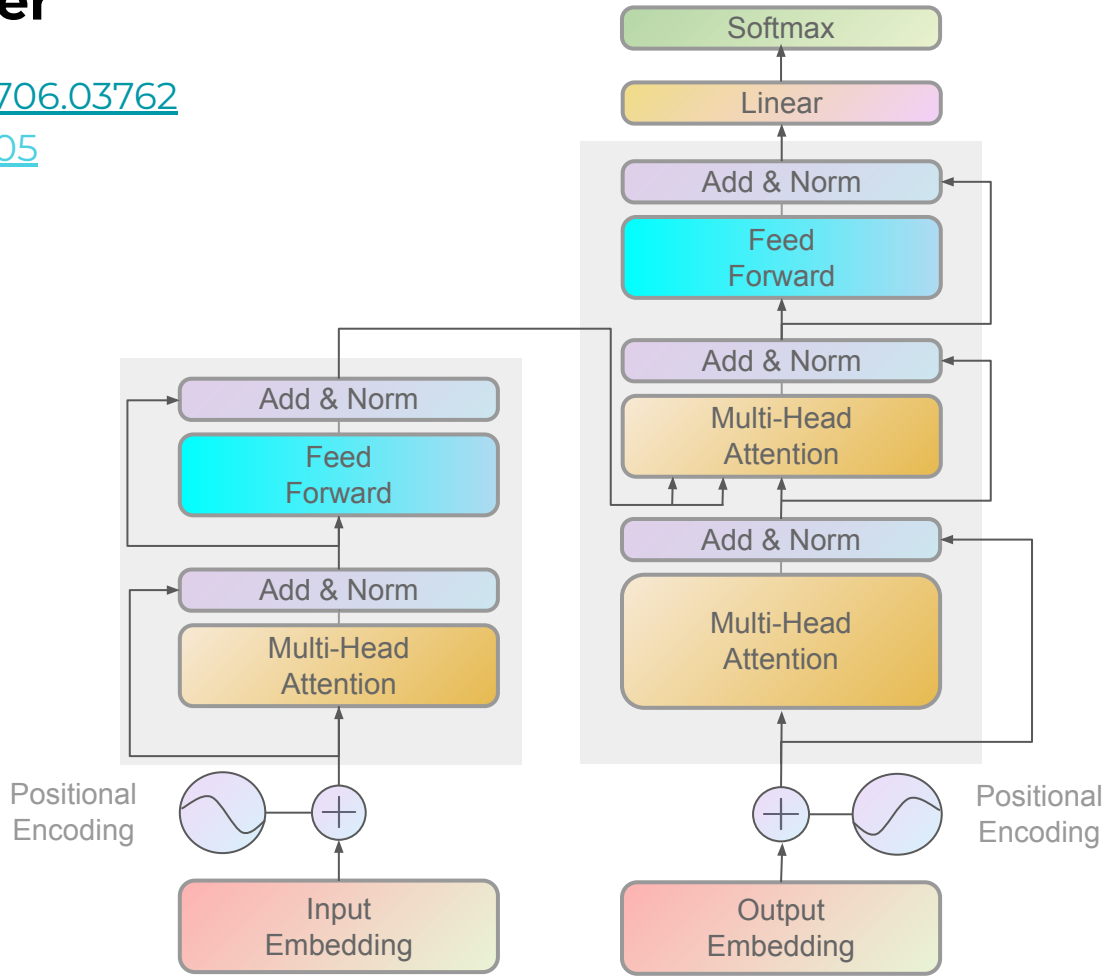
Transformer [Positional embedding]



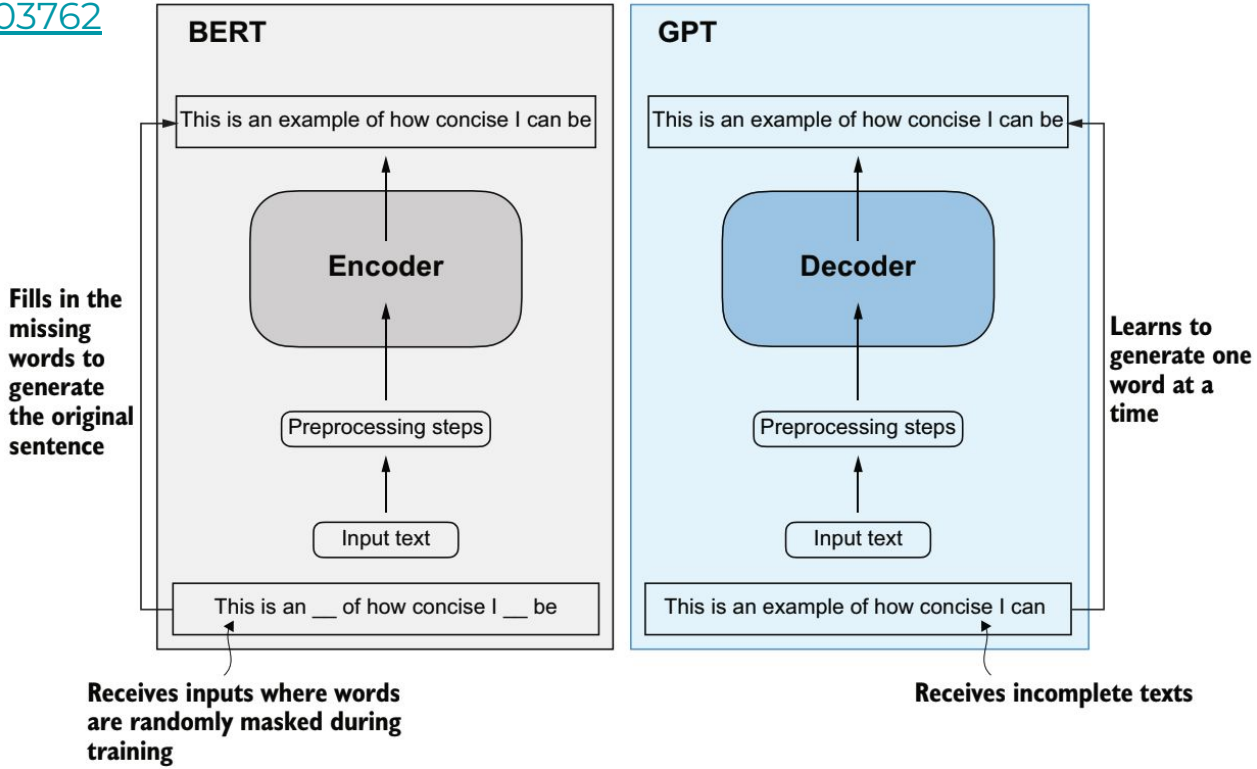
https://visionbook.mit.edu/transformers.html

Transformer

Transformer: 1706.03762
BERT: 1810.04805
GPT: 1810

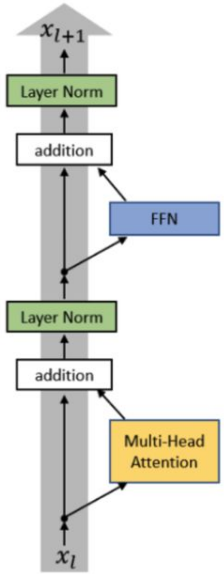


Transformer: 1706.03762
BERT: 1810.04805
GPT: 1810



Sebastian Raschka “Build a Large Language Model” (From Scratch)

Latest practices [Pre-vs-post norm]



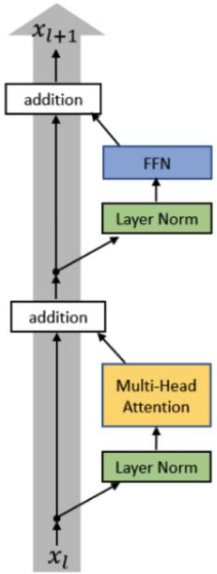


Figure from Xiong 2020

Post-LN Transformer	Pre-LN Transformer
$x_{l,i}^{post,1} = \text{MultiHeadAtt}(x_{l,i}^{post}, [x_{l,1}^{post}, \cdots, x_{l,n}^{post}])$	$x_{l,i}^{pre,1} = \text{LayerNorm}(x_{l,i}^{pre})$
$x_{l,i}^{post,2} = x_{l,i}^{post} + x_{l,i}^{post,1}$	$x_{l,i}^{pre,2} = \text{MultiHeadAtt}(x_{l,i}^{pre,1}, [x_{l,1}^{pre,1}, \cdots, x_{l,n}^{pre,1}])$
$x_{l,i}^{post,3} = \text{LayerNorm}(x_{l,i}^{post,2})$	$x_{l,i}^{pre,3} = x_{l,i}^{pre} + x_{l,i}^{pre,2}$
$x_{l,i}^{post,4} = \text{ReLU}(x_{l,i}^{post,3} W^{1,l} + b^{1,l}) W^{2,l} + b^{2,l}$	$x_{l,i}^{pre,4} = \text{LayerNorm}(x_{l,i}^{pre,3})$
$x_{l,i}^{post,5} = x_{l,i}^{post,3} + x_{l,i}^{post,4}$	$x_{l,i}^{pre,5} = \text{ReLU}(x_{l,i}^{pre,4} W^{1,l} + b^{1,l}) W^{2,l} + b^{2,l}$
$x_{l+1,i}^{post} = \text{LayerNorm}(x_{l,i}^{post,5})$	$x_{l+1,i}^{pre} = x_{l,i}^{pre,5} + x_{l,i}^{pre,3}$
	Final LayerNorm: $x_{Final,i}^{pre} \leftarrow \text{LayerNorm}(x_{L+1,i}^{pre})$

Latest practices [RMSNorm]

Original transformer: **LayerNorm** – normalizes the mean and variance across d_{model}

$$y = \frac{x - \mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}} * \gamma + \beta$$

Many modern LMs: **RMSNorm** – does not subtract mean or add a bias term

$$y = \frac{x}{\sqrt{||x||_2^2 + \varepsilon}} * \gamma$$

Latest practices [RMSNorm]

Differences from LayerNorm

- **No Mean Subtraction**: LayerNorm subtracts the mean and divides by the standard deviation, while RMSNorm only divides by the RMS value, skipping the mean computation.
- **Efficiency**: By avoiding mean computation, RMSNorm is computationally cheaper, especially for large feature dimensions.
- **Performance**: RMSNorm has been shown to perform comparably or better in some Transformer-based models (e.g., as used in models like LLaMA).

doc_link

RMSNorm

CLASS torch.nn.RMSNorm(*normalized_shape*, *eps=None*, *elementwise_affine=True*, *device=None*, *dtype=None*) [SOURCE]

Applies Root Mean Square Layer Normalization over a mini-batch of inputs.

This layer implements the operation as described in the paper Root Mean Square Layer Normalization

$$y_i = \frac{x_i}{\mathrm{RMS}(x)} * \gamma_i, \quad \text{where} \quad \mathrm{RMS}(x) = \sqrt{\epsilon + \frac{1}{n}\sum_{i=1}^{n} x_i^2}$$

The RMS is taken over the last `D` dimensions, where `D` is the dimension of `normalized_shape`. For example, if `normalized_shape` is `(3, 5)` (a 2-dimensional shape), the RMS is computed over the last 2 dimensions of the input.

Parameters

- **normalized_shape** (*int* or *list* or *torch.Size*) – input shape from an expected input of size

  $$[* \times \text{normalized\_shape}[0] \times \text{normalized\_shape}[1] \times \ldots \times \text{normalized\_shape}[-1]]$$

  If a single integer is used, it is treated as a singleton list, and this module will normalize over the last dimension which is expected to be of that specific size.
- **eps** (*Optional*[*float*]) – a value added to the denominator for numerical stability. Default: `torch.finfo(x.dtype).eps()`
- **elementwise_affine** (*bool*) – a boolean value that when set to `True`, this module has learnable per-element affine parameters initialized to ones (for weights). Default: `True`.

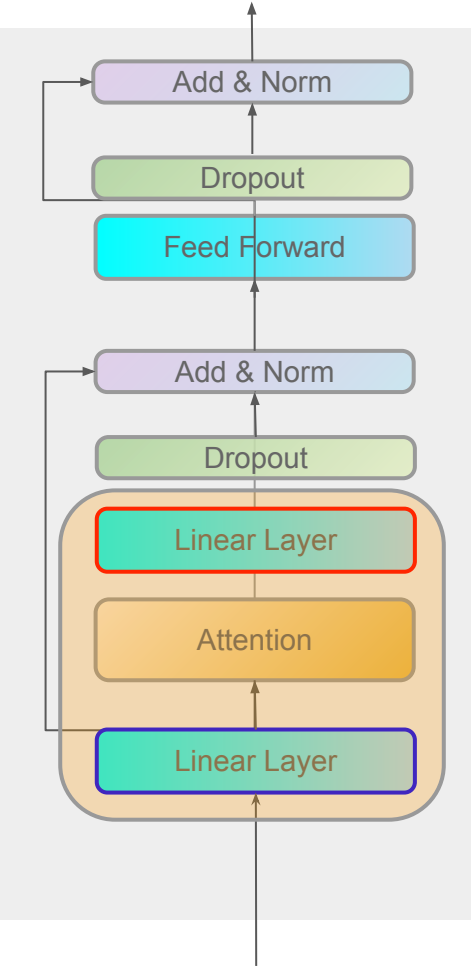
Latest practices [drop bias terms]

$$FC(X) = Linear_{[\mathbf{W_{out}}, \mathbf{b_{out}}]}(\sigma(Linear_{[\mathbf{W_{in}}, \mathbf{b_{in}}]}(X)))$$

$$MultiHead(\mathbf{X}, \mathbf{X}) = Linear_{[\mathbf{W^O}, \mathbf{b^O}]}\left(\underset{i \in [A]}{concat}\left[\mathbf{H^i}\right]\right)$$

$$Attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) \cdot V$$

$$\mathbf{H^i} = Attention(Linear_{[\mathbf{W_Q^{(i)}}, \mathbf{b_Q^{(i)}}]}(X), Linear_{[\mathbf{W_K^{(i)}}, \mathbf{b_K^{(i)}}]}(X), Linear_{[\mathbf{W_V^{(i)}}, \mathbf{b_V^{(i)}}]}(X))$$



Latest practices [ROPE]

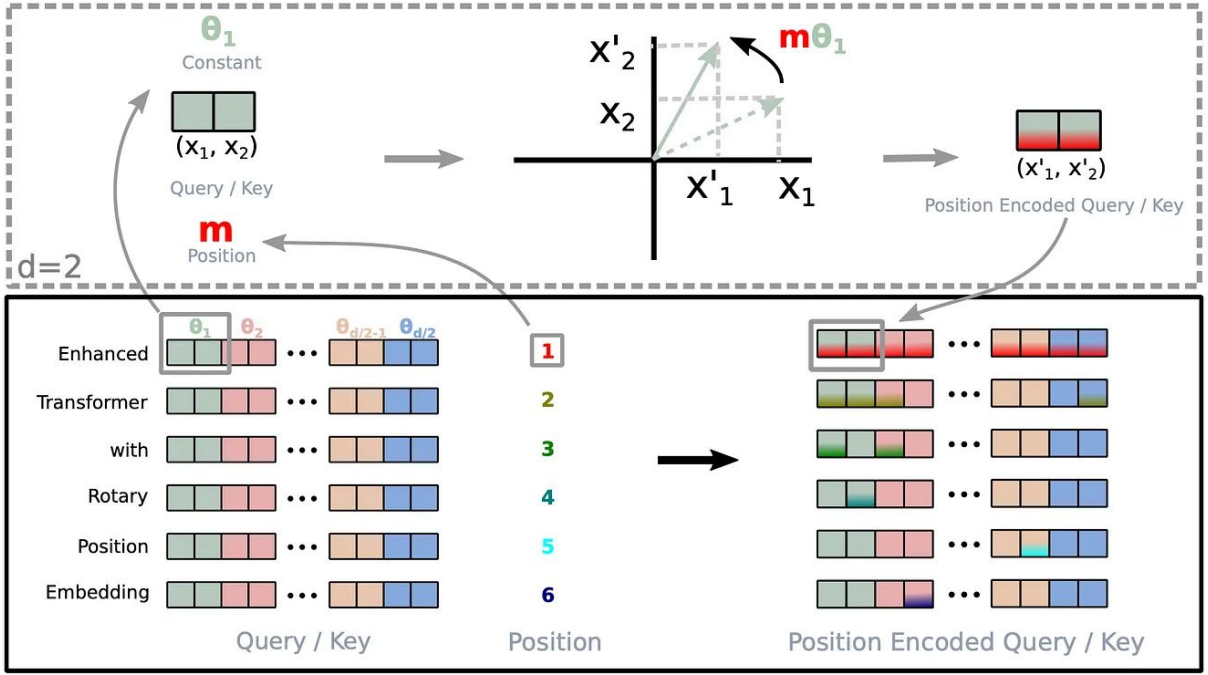
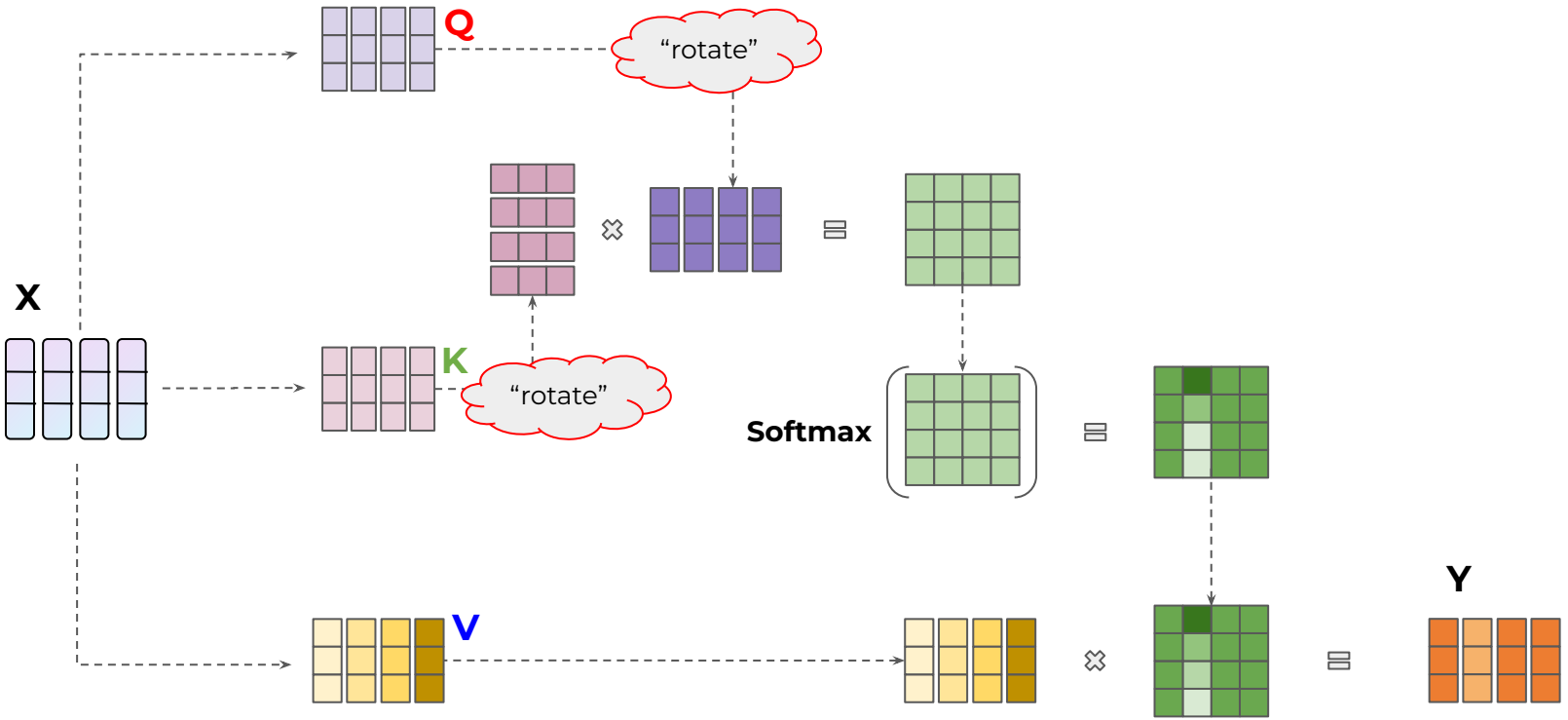


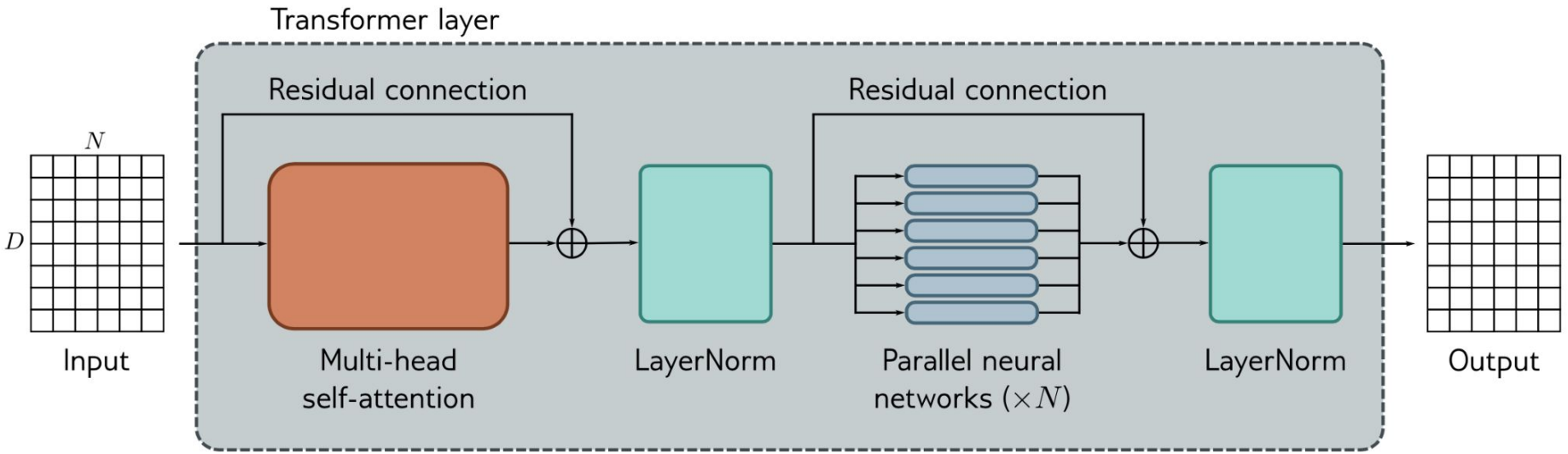
Figure 1: Implementation of Rotary Position Embedding(RoPE).

[2104.09864] RoFormer: Enhanced Transformer with Rotary Position Embedding
RoPE: A Detailed Guide to Rotary Position Embedding in Modern LLMs | by Allen Liang | Medium

Latest practices [ROPE]



Latest practices [head, tail]



Multi-head **latent** attention

RMSNorm

MoE

4 Conclusions

We have extended the GLU family of layers and proposed their use in Transformer. In a transfer-learning setup, the new variants seem to produce better perplexities for the de-noising objective used in pre-training, as well as better results on many downstream language-understanding tasks. These architectures are simple to implement, and have no apparent computational drawbacks. We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.

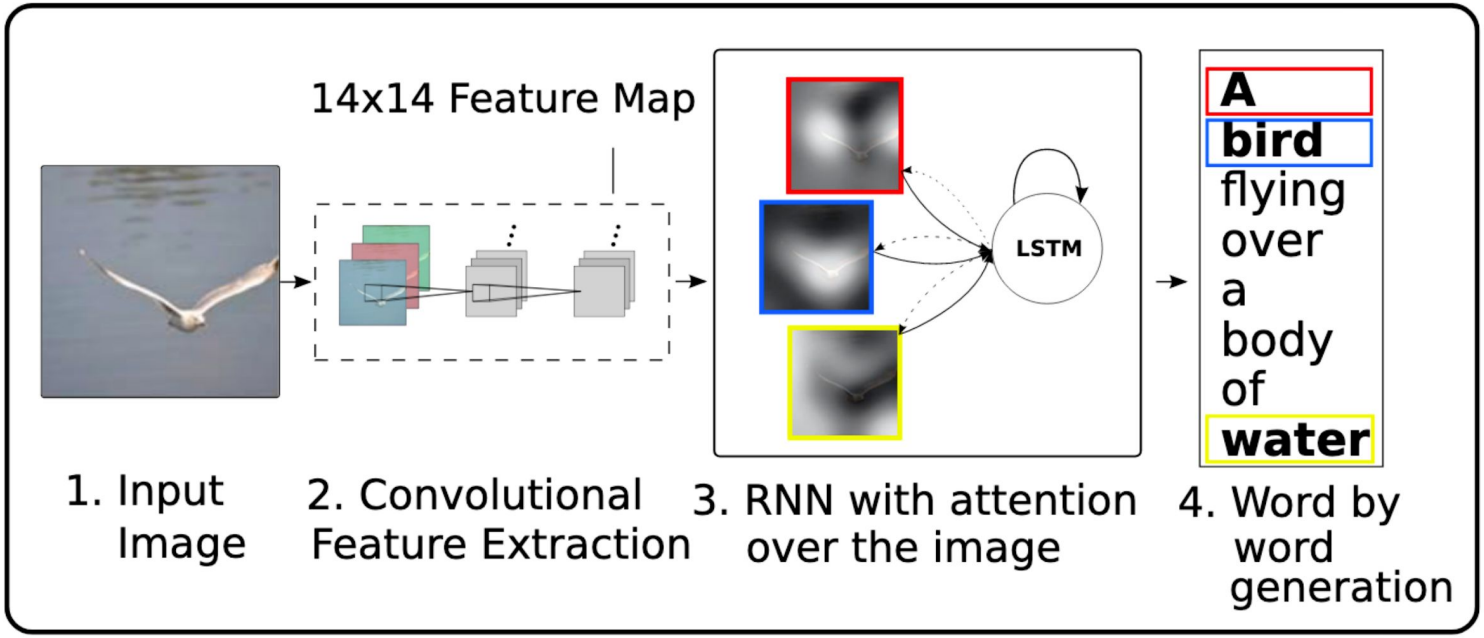
2002.05202

Content of today's lecture

- Attention Intro
  - Seq2seq Attention
  - Self-Attention
  - Transformer

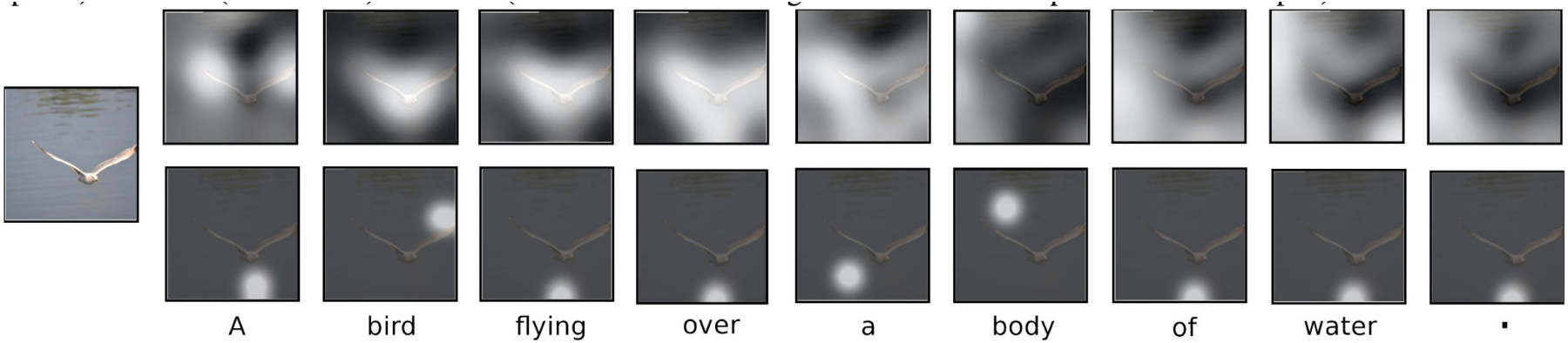
- Attention in CV
  - SE
  - ViT
  - latest developments

Attention [seq2seq, visual]



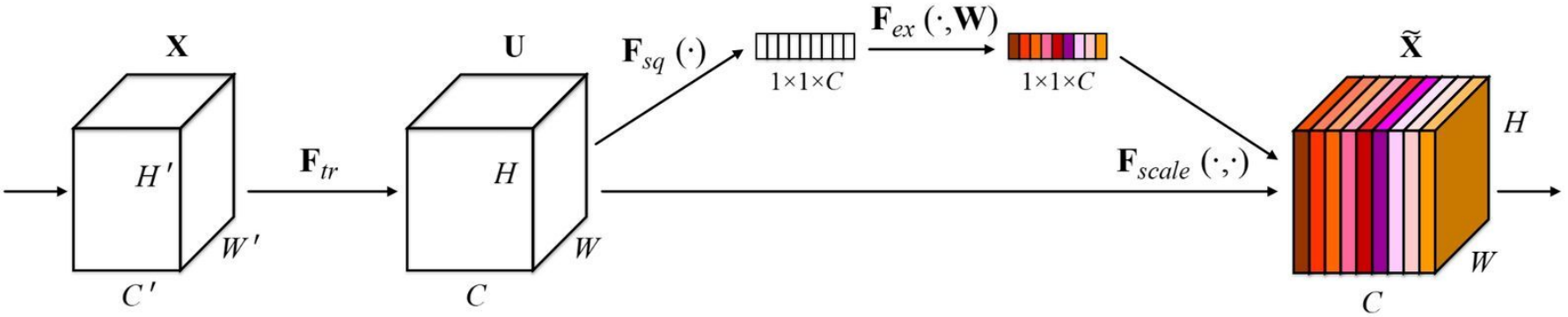
1502.03044

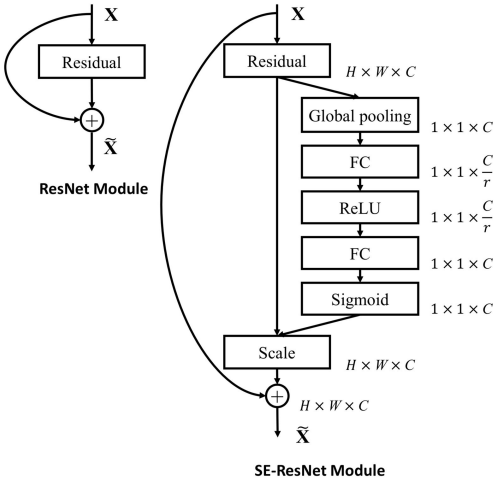
Attention [seq2seq, visual]



1502.03044

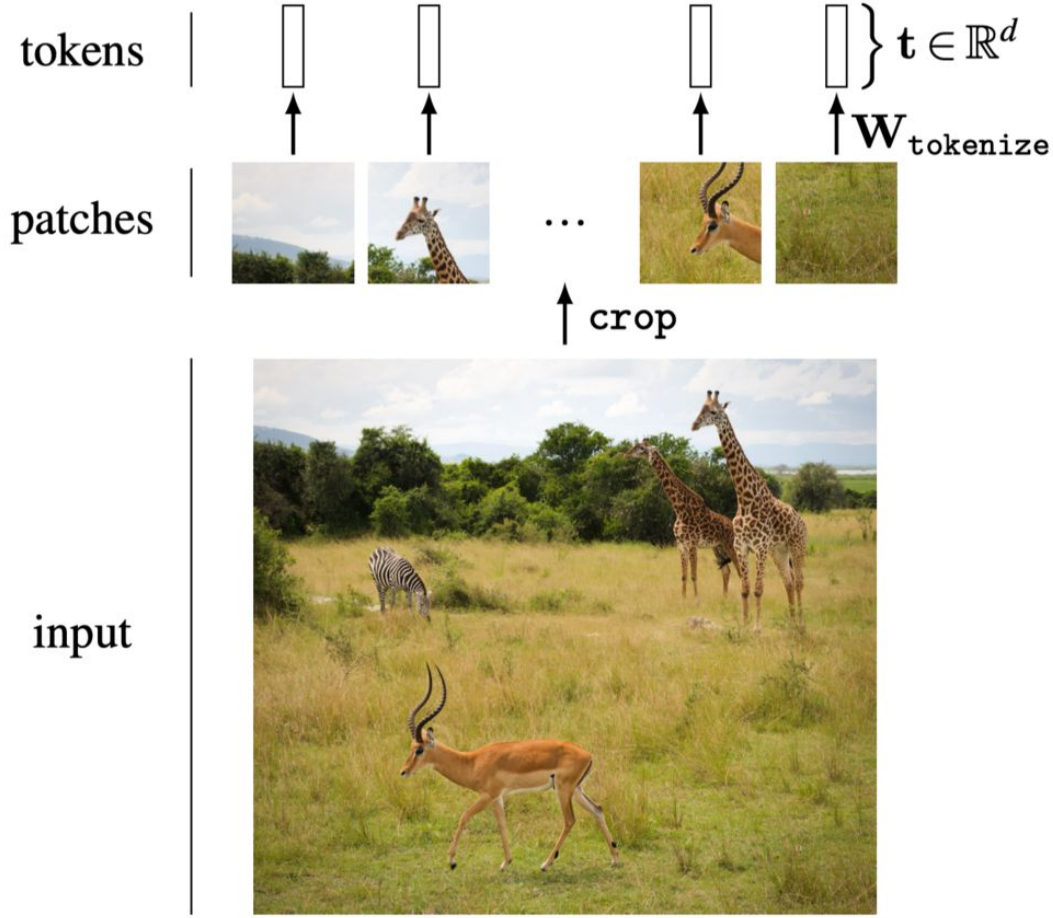
Squeeze-and-Excitation (SE) Block

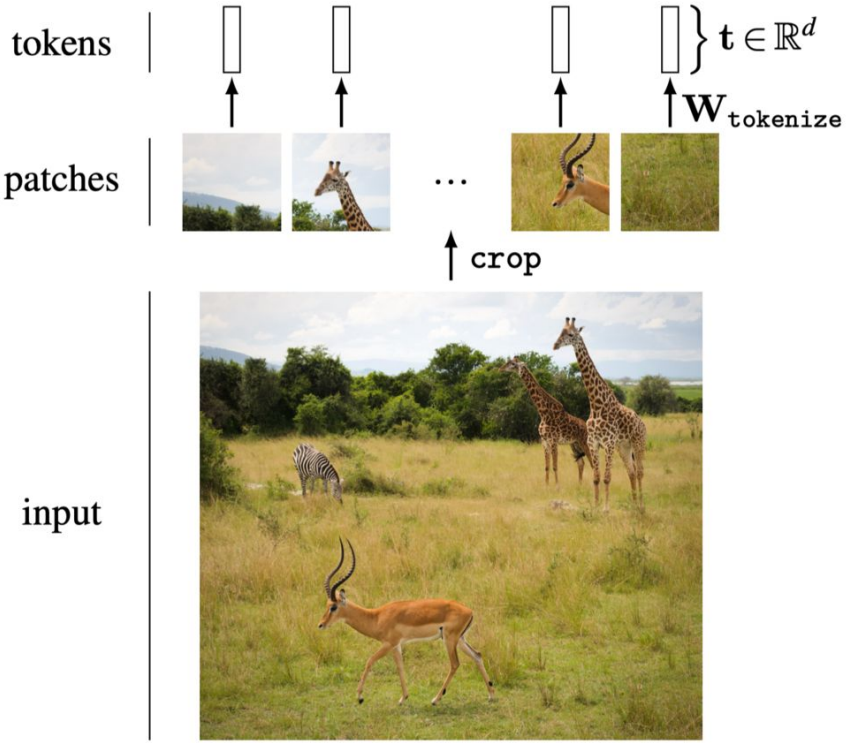


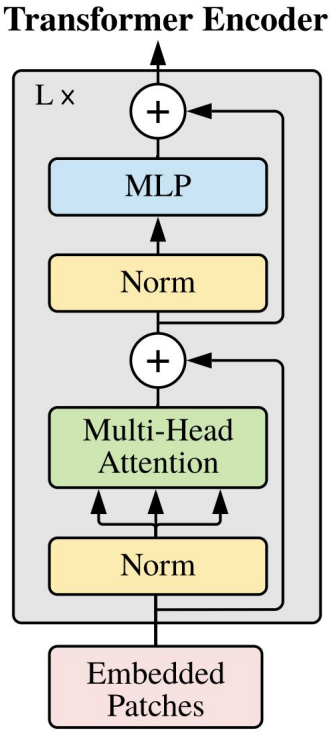


1709.01507

ViT

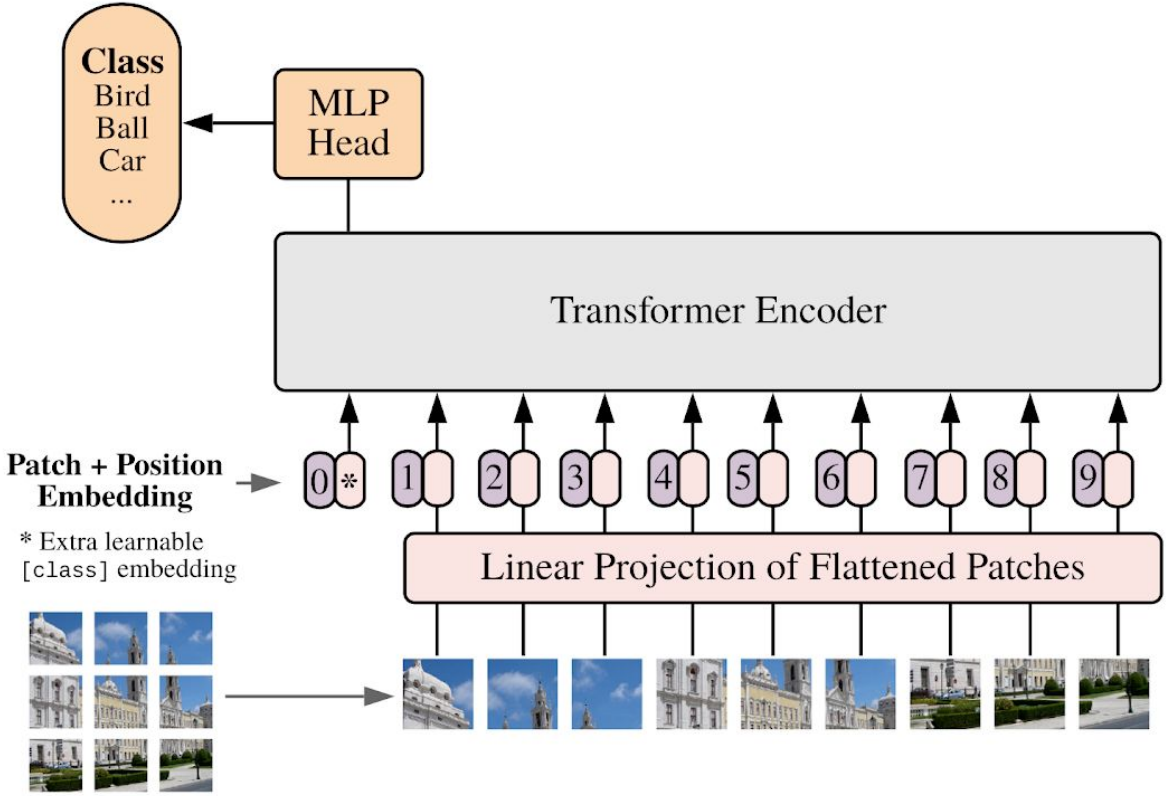

tokens
$\mathbf{t} \in \mathbb{R}^d$
$\mathbf{W}_{\texttt{tokenize}}$
patches
...
crop
input





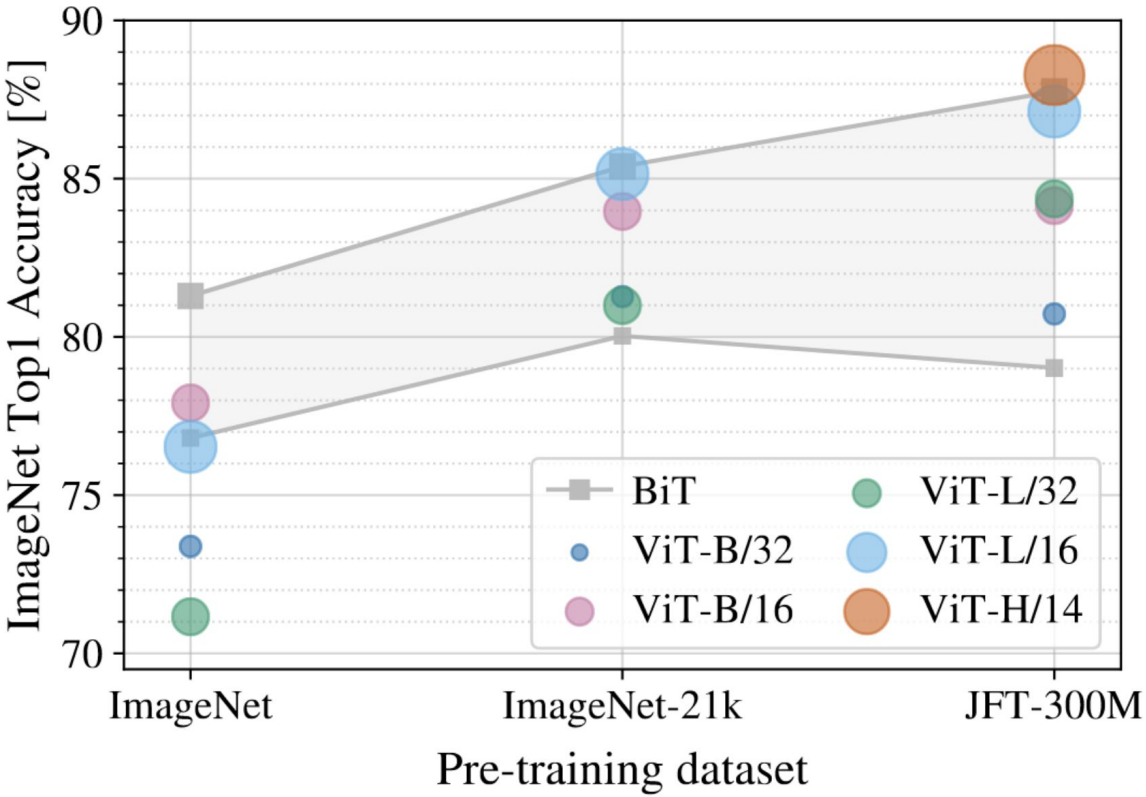
[2010.11929] An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

ViT



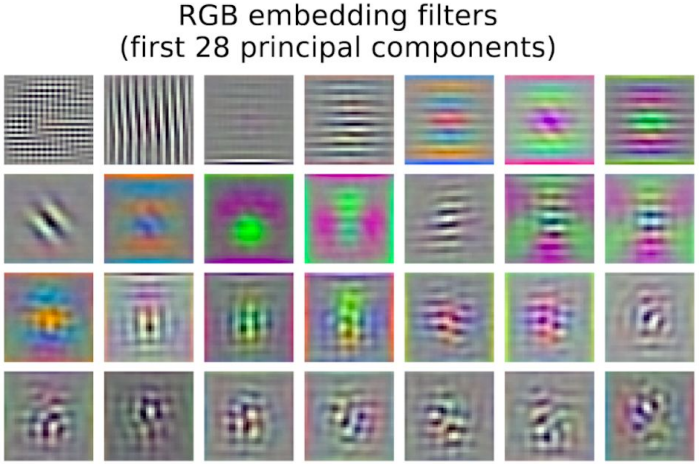
2010.11929

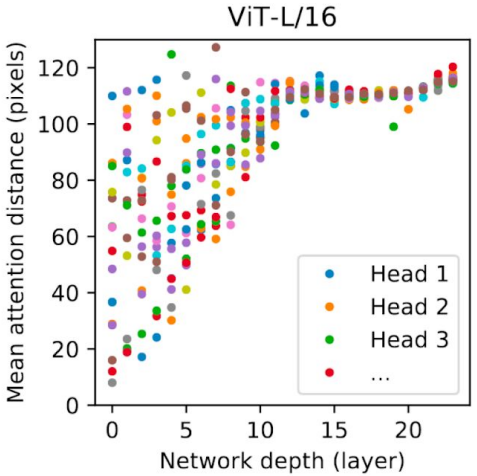
ViT



2010.11929

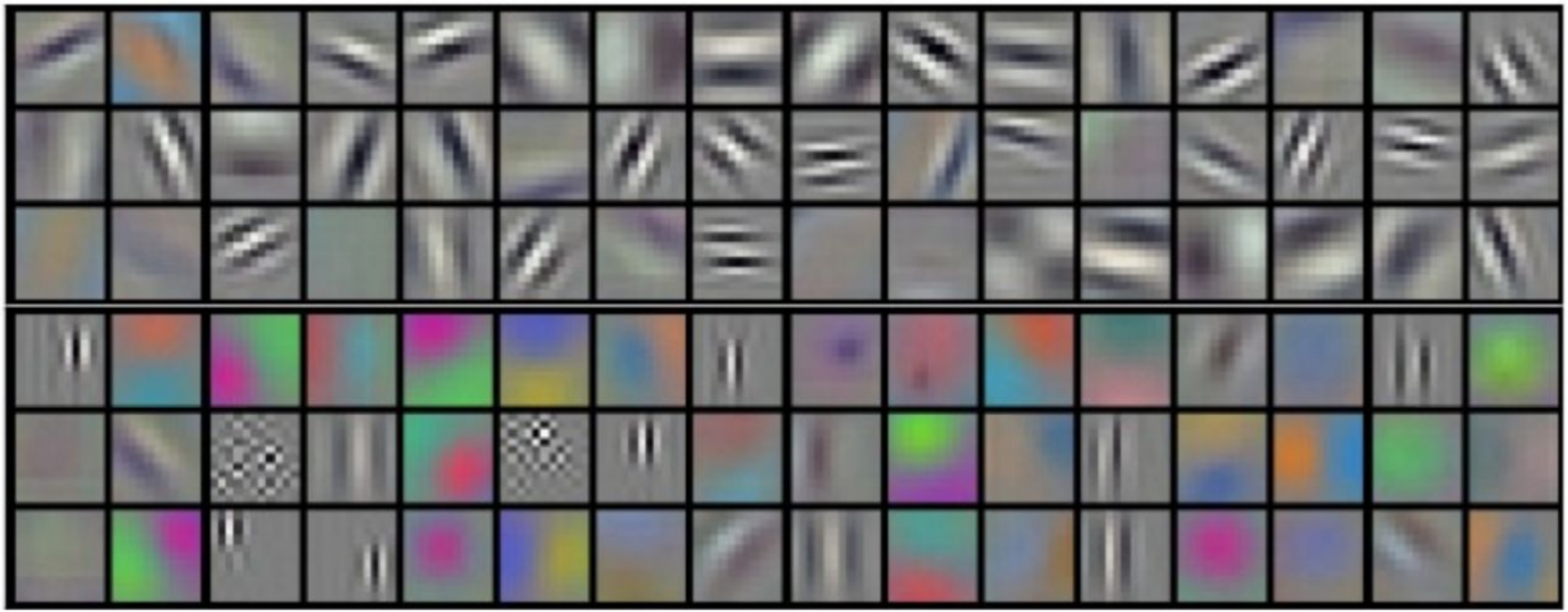
ViT





2010.11929

CNNs



DINOv3

Oriane Siméoni* **Huy V. Vo*** **Maximilian Seitzer*** **Federico Baldassarre*** **Maxime Oquab***
Cijo Jose **Vasil Khalidov** **Marc Szafraniec** **Seungeun Yi** **Michaël Ramamonjisoa**
Francisco Massa **Daniel Haziza** **Luca Wehrstedt** **Jianyuan Wang**
Timothée Darcet **Théo Moutakanni** **Leonel Sentana** **Claire Roberts**
Andrea Vedaldi **Jamie Tolan** **John Brandt**[1] **Camille Couprie**
Julien Mairal[2] **Hervé Jégou** **Patrick Labatut** **Piotr Bojanowski**

Meta AI Research [1]*WRI* [2]*Inria*

corresponding authors: {osimeoni,huyvvo,seitzer,baldassarre,qas}@meta.com

Abstract

Self-supervised learning holds the promise of eliminating the need for manual data annotation, enabling models to scale effortlessly to massive datasets and larger architectures. By not being tailored to specific tasks or domains, this training paradigm has the potential to learn visual representations from diverse sources, ranging from natural to aerial images—using a single algorithm. This technical report introduces DINOv3, a major milestone toward realizing this vision by leveraging simple yet effective strategies. First, we leverage the benefit of scaling both dataset and model size by careful data preparation, design, and optimization. Second, we introduce a new method called Gram anchoring, which effectively addresses the known yet unsolved issue of dense feature maps degrading during long training schedules. Finally, we apply post-hoc strategies that further enhance our models' flexibility with respect to resolution, model size, and alignment with text. As a result, we present a versatile vision foundation model that outperforms the specialized state of the art across a broad range of settings, without fine-tuning. DINOv3 produces high-quality dense features that achieve outstanding performance on various vision tasks, significantly surpassing previous self- and weakly-supervised foundation models. We also share the DINOv3 suite of vision models, designed to advance the state of the art on a wide spectrum of tasks and data by providing scalable solutions for diverse resource constraints and deployment scenarios.

1 Introduction

Foundation models have become a central building block in modern computer vision, enabling broad generalization across tasks and domains through a single, reusable model. Self-supervised learning (SSL) is a powerful approach for training such models, by learning directly from raw pixel data and leveraging the nat-

VIT [vit-base-patch16]

```
ViTForImageClassification(
  (vit): ViTModel(
    (embeddings): ViTEmbeddings(
      (patch embeddings): ViTPatchEmbeddings(
        (projection): Conv2d(3, 768, kernel_size=(16, 16), stride=(16, 16))
      )
      (dropout): Dropout(p=0.0, inplace=False)
    )
    (encoder): ViTEncoder(
      (layer): ModuleList(
        (0-11): 12 x ViTLayer(
          (attention): ViTAttention(
            (attention): ViTSelfAttention(
              (query): Linear(in features=768, out features=768, bias=True)
              (key): Linear(in features=768, out features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
            )
            (output): ViTSelfOutput(
              (dense): Linear(in features=768, out features=768, bias=True)
              (dropout): Dropout(p=0.0, inplace=False)
            )
          )
          (intermediate): ViTIntermediate(
            (dense): Linear(in features=768, out features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): ViTOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
          )
          (layernorm before): LayerNorm((768,), eps=1e-12, elementwise affine=True)
          (layernorm_after): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        )
      )
    )
    (layernorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
  )
  (classifier): Linear(in_features=768, out_features=1000, bias=True)
)
```

ViT [DINOv3]

```
DINOv3ViTModel(
  (embeddings): DINOv3ViTEmbeddings(
    (patch_embeddings): Conv2d(3, 768, kernel_size=(16, 16), stride=(16, 16))
  )
  (rope_embeddings): DINOv3ViTRopePositionEmbedding()
  (layer): ModuleList(
    (0-11): 12 x DINOv3ViTLayer(
      (norm1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
      (attention): DINOv3ViTAttention(
        (k_proj): Linear(in_features=768, out_features=768, bias=False)
        (v_proj): Linear(in_features=768, out_features=768, bias=True)
        (q_proj): Linear(in_features=768, out_features=768, bias=True)
        (o_proj): Linear(in_features=768, out_features=768, bias=True)
      )
      (layer_scale1): DINOv3ViTLayerScale()
      (drop_path): Identity()
      (norm2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
      (mlp): DINOv3ViTMLP(
        (up_proj): Linear(in_features=768, out_features=3072, bias=True)
        (down_proj): Linear(in_features=3072, out_features=768, bias=True)
        (act_fn): GELUActivation()
      )
      (layer_scale2): DINOv3ViTLayerScale()
    )
  )
  (norm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
)
```

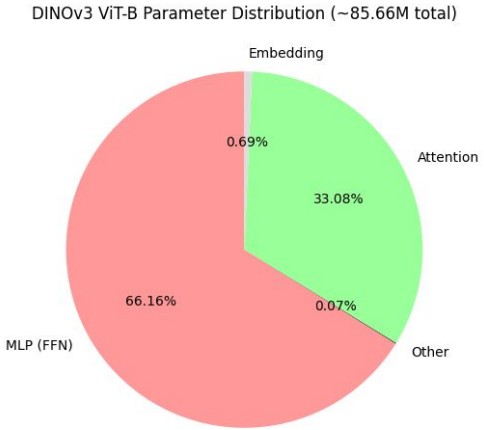
[HF] modular_dinov3_vit.py

ViT [DINOv3]

```python
class DINOv3ViTLayer(GradientCheckpointingLayer):
    def __init__(self, config: DINOv3ViTConfig):
        super().__init__()

        # 1) First Norm before attention
        self.norm1 = nn.LayerNorm(config.hidden_size, eps=config.layer_norm_eps)

        # 2) Attention mechanism
        self.attention = DINOv3ViTAttention(config)
        self.layer_scale1 = DINOv3ViTLayerScale(config)
        self.drop_path = DINOv3ViTDropPath(config.drop_path_rate) if config.drop_pat

        # 3) Second Norm before MLP
        self.norm2 = nn.LayerNorm(config.hidden_size, eps=config.layer_norm_eps)

        # 4) Feed-forward network
        if config.use_gated_mlp:
            self.mlp = DINOv3ViTGatedMLP(config)
        else:
            self.mlp = DINOv3ViTMLP(config)
        self.layer_scale2 = DINOv3ViTLayerScale(config)
```

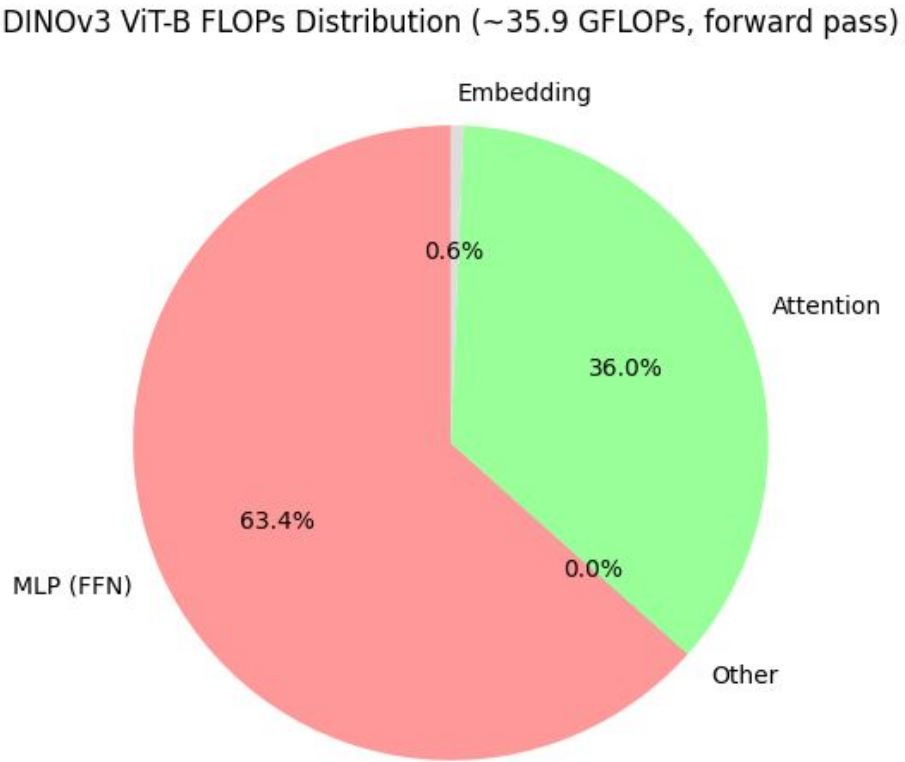
[HF] modular_dinov3_vit.py

ViT [DINOv3, base, memory]

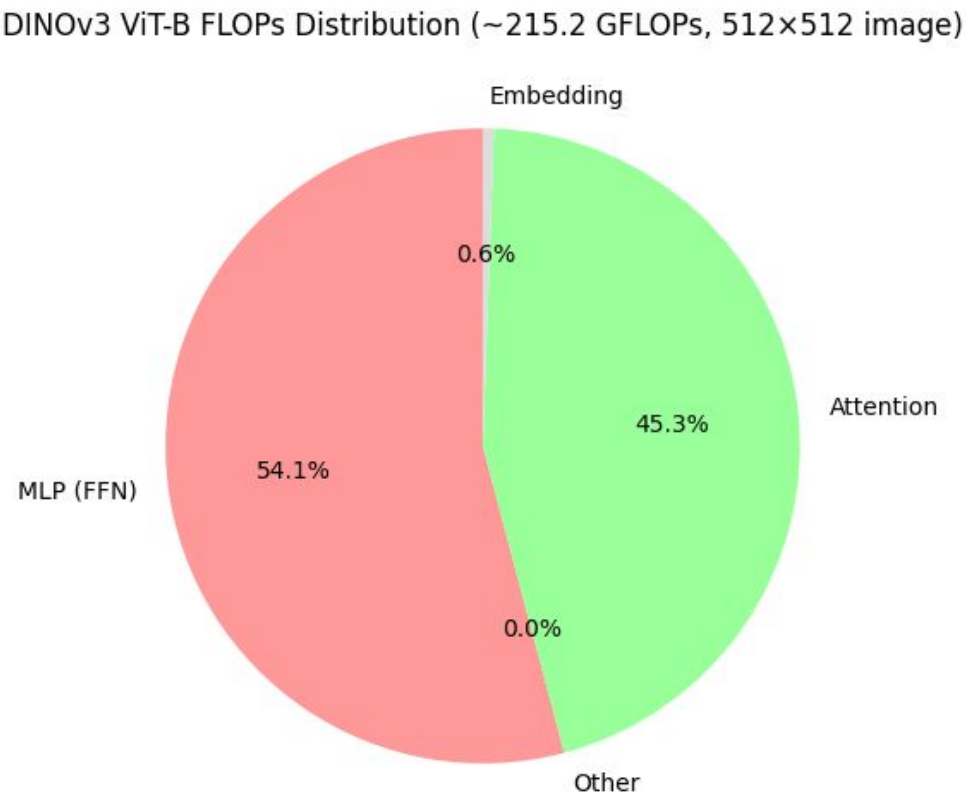


Component	Params	% of total (~85.66M)
Embeddings (CLS + mask + register + patch Conv2d)	595,200	0.695 %
Attention projections (Q, K, V, O across 12 layers)	28,339,200	33.10 %
MLP (up_proj + down_proj across 12 layers)	56,669,184	66.17 %
LayerNorms in layers (norm1 + norm2 × 12)	36,864	0.043 %
LayerScale in layers (layer_scale1 + layer_scale2 × 12)	18,432	0.022 %
Final LayerNorm	1,536	0.0018 %

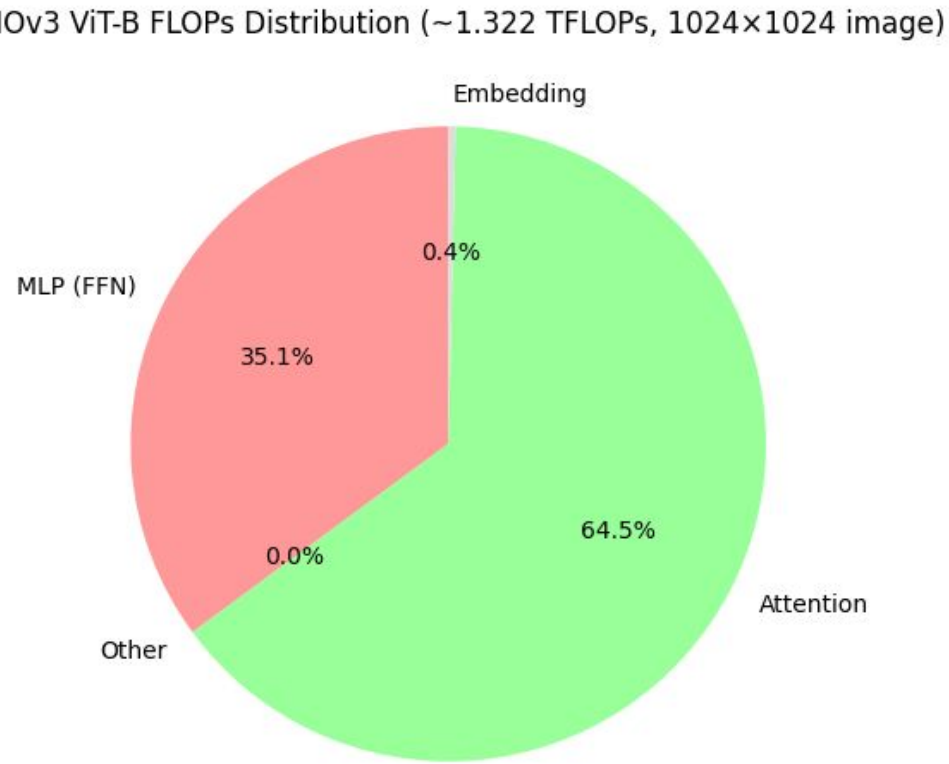
ViT [DINOv3, base, compute]



ViT [DINOv3, base, compute]



ViT [DINOv3, base, compute]



DINOv3 models

Model	#Params	Inference GFLOPs	
		Res. 256	Res. 512
CNX-Tiny	29M	5	20
CNX-Small	50M	11	46
CNX-Base	89M	20	81
CNX-Large	198M	38	152
ViT-S	21M	12	63
ViT-S+	29M	16	79
ViT-B	86M	47	216
ViT-L	300M	163	721
ViT-H+	840M	450	1903
ViT-7B	6716M	3550	14515

(a) DINOv3 family of models.

2508.10104

ConvNEXT

	output size	• ResNet-50	• ConvNeXt-T
stem	56×56	7×7, 64, stride 2 3×3 max pool, stride 2	4×4, 96, stride 4
res2	56×56	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} d7\times7, 96 \\ 1\times1, 384 \\ 1\times1, 96 \end{bmatrix} \times 3$
res3	28×28	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} d7\times7, 192 \\ 1\times1, 768 \\ 1\times1, 192 \end{bmatrix} \times 3$
res4	14×14	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} d7\times7, 384 \\ 1\times1, 1536 \\ 1\times1, 384 \end{bmatrix} \times 9$
res5	7×7	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} d7\times7, 768 \\ 1\times1, 3072 \\ 1\times1, 768 \end{bmatrix} \times 3$
FLOPs		4.1×10^9	4.5×10^9
# params.		25.6×10^6	28.6×10^6

- ConvNeXt-T: $C = (96, 192, 384, 768), B = (3, 3, 9, 3)$
- ConvNeXt-S: $C = (96, 192, 384, 768), B = (3, 3, 27, 3)$
- ConvNeXt-B: $C = (128, 256, 512, 1024), B = (3, 3, 27, 3)$
- ConvNeXt-L: $C = (192, 384, 768, 1536), B = (3, 3, 27, 3)$

convnext_tiny — Torchvision main documentation

[2201.03545] A ConvNet for the 2020s

ConvNEXT

```
DINOv3ConvNextModel(
 (stages): ModuleList(
   (0): DINOv3ConvNextStage(
     (downsample layers): ModuleList(
       (0): Conv2d(3, 96, kernel size=(4, 4), stride=(4, 4))
       (1): DINOv3ConvNextLayerNorm((96,), eps=1e-06, elementwise_affine=True)
     )
     (layers): ModuleList(
       (0-2): 3 x DINOv3ConvNextLayer(
         (depthwise conv): Conv2d(96, 96, kernel size=(7, 7), stride=(1, 1), padding=(3, 3), groups=96)
         (layer norm): DINOv3ConvNextLayerNorm((96,), eps=1e-06, elementwise affine=True)
         (pointwise conv1): Linear(in features=96, out_features=384, bias=True)
         (activation fn): GELUActivation()
         (pointwise conv2): Linear(in_features=384, out_features=96, bias=True)
         (drop_path): Identity()
       )
     )
   )
   (1): ...
   (2): ...
   (3): ...
 )
 (layer norm): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
 (pool): AdaptiveAvgPool2d(output_size=1)
)
```

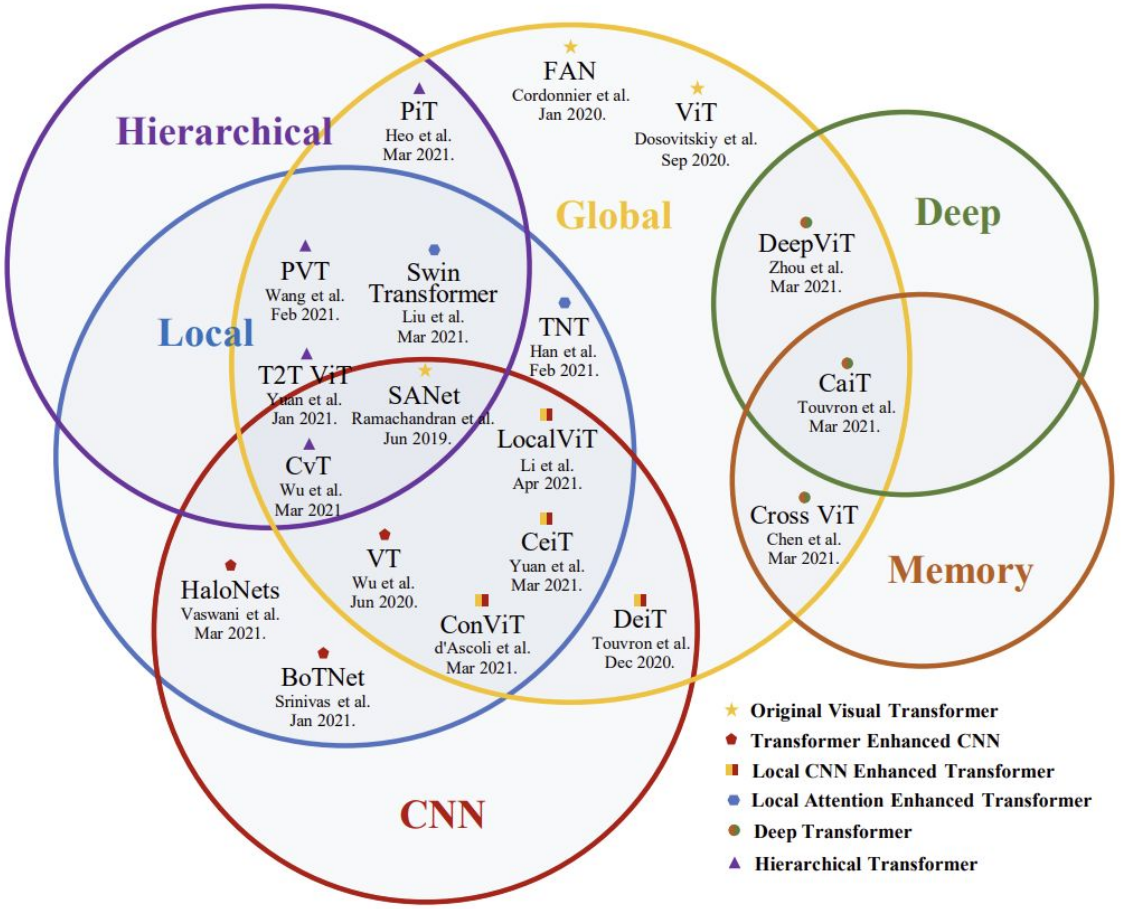
[2201.03545] A ConvNet for the 2020s

ConvNEXT

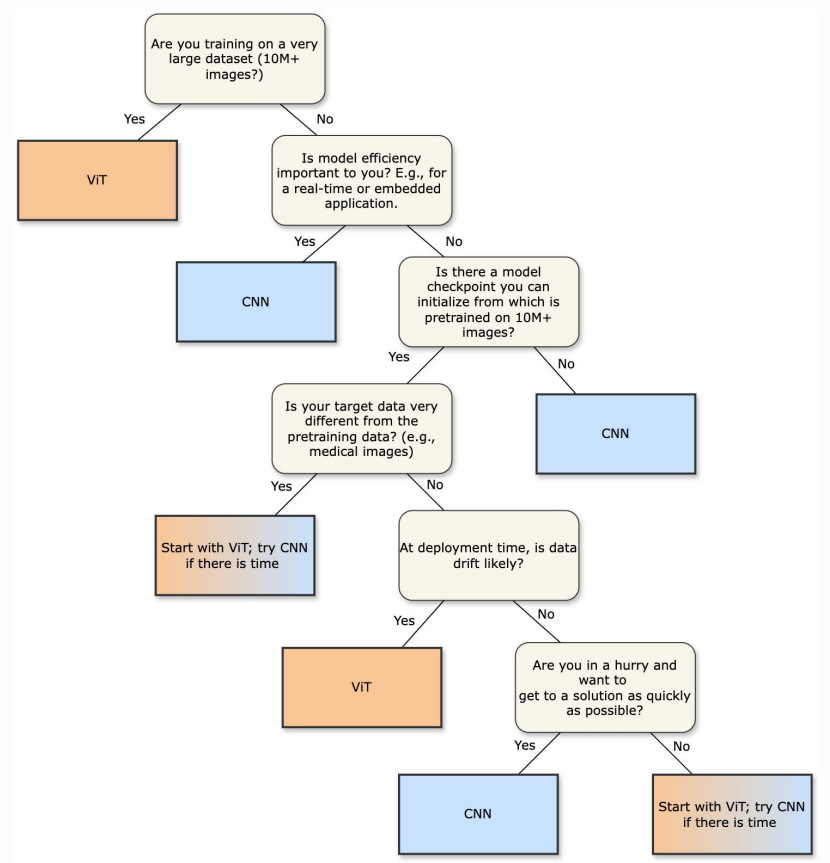
```
downsample layers.0                  : [(1, 3, 224, 224)] -> (1, 128, 56, 56)
downsample_layers.1                  : [(1, 128, 56, 56)] -> (1, 128, 56, 56)

layers.0.depthwise conv              : [(1, 128, 56, 56)] -> (1, 128, 56, 56)
layers.0.layer norm                  : [(1, 56, 56, 128)] -> (1, 56, 56, 128)
layers.0.pointwise conv1             : [(1, 56, 56, 128)] -> (1, 56, 56, 512)
layers.0.activation fn               : [(1, 56, 56, 512)] -> (1, 56, 56, 512)
layers.0.pointwise conv2             : [(1, 56, 56, 512)] -> (1, 56, 56, 128)
layers.0.drop_path                   : [(1, 128, 56, 56)] -> (1, 128, 56, 56)

layers.1.depthwise conv              : [(1, 128, 56, 56)] -> (1, 128, 56, 56)
layers.1.layer norm                  : [(1, 56, 56, 128)] -> (1, 56, 56, 128)
layers.1.pointwise conv1             : [(1, 56, 56, 128)] -> (1, 56, 56, 512)
layers.1.activation fn               : [(1, 56, 56, 512)] -> (1, 56, 56, 512)
layers.1.pointwise conv2             : [(1, 56, 56, 512)] -> (1, 56, 56, 128)
layers.1.drop_path                   : [(1, 128, 56, 56)] -> (1, 128, 56, 56)

layers.2.depthwise conv              : [(1, 128, 56, 56)] -> (1, 128, 56, 56)
layers.2.layer norm                  : [(1, 56, 56, 128)] -> (1, 56, 56, 128)
layers.2.pointwise conv1             : [(1, 56, 56, 128)] -> (1, 56, 56, 512)
layers.2.activation fn               : [(1, 56, 56, 512)] -> (1, 56, 56, 512)
layers.2.pointwise conv2             : [(1, 56, 56, 512)] -> (1, 56, 56, 128)
layers.2.drop_path                   : [(1, 128, 56, 56)] -> (1, 128, 56, 56)
```

[2201.03545] A ConvNet for the 2020s

Hybrids


Hierarchical
Global
Deep
Local
CNN
Memory
PiT
Heo et al.
Mar 2021.
FAN
Cordonnier et al.
Jan 2020.
ViT
Dosovitskiy et al.
Sep 2020.
DeepViT
Zhou et al.
Mar 2021.
PVT
Wang et al.
Feb 2021.
Swin
Transformer
Liu et al.
Mar 2021.
TNT
Han et al.
Feb 2021.
CaiT
Touvron et al.
Mar 2021.
T2T ViT
Yuan et al.
Jan 2021.
SANet
Ramachandran et al.
Jun 2019.
LocalViT
Li et al.
Apr 2021.
CvT
Wu et al.
Mar 2021
Cross ViT
Chen et al.
Mar 2021.
CeiT
Yuan et al.
Mar 2021.
VT
Wu et al.
Jun 2020.
HaloNets
Vaswani et al.
Mar 2021.
ConViT
d'Ascoli et al.
Mar 2021.
DeiT
Touvron et al.
Dec 2020.
BoTNet
Srinivas et al.
Jan 2021.
Original Visual Transformer
Transformer Enhanced CNN
Local CNN Enhanced Transformer
Local Attention Enhanced Transformer
Deep Transformer
Hierarchical Transformer

CNN vs ViT



A decision tree for deciding between CNN and ViT for real-world projects

[CNN vs. Vision Transformer: A Practitioner's Guide to Selecting the Right Model]